



Hochschule für Technik
und Wirtschaft Berlin

University of Applied Sciences

Explainable Artificial Intelligence in GraphCast: A Study on the Forecast Transparency of a Deep Learning-Based Weather Model

Thesis for the attainment of the academic degree:
Bachelor of Science (B.Sc.)

at the
University of Applied Sciences for Engineering and Economics (HTW) Berlin
Faculty 4: Computer Science, Communication and Business
Degree Program: Applied Computer Science

1st Reviewer: Prof. Dr. Christian Herta
2nd Reviewer: Dr. Christian Krumnow

Submitted by: Esta Kempf

06.08.2025

Declaration of independent work

I herewith declare that I wrote and composed this thesis on my own with no other sources or aids than cited. Used information from open literature has been identified correctly according to established academic citation rules. I confirm that this study has not been previously submitted in an identical or similar form to any Institutions or Universities. I am aware that the University is entitled to revoke the degree awarded on the basis of this thesis in case of copyright violation as well as an attempted fraud.

06.08.2025

A handwritten signature in black ink, appearing to be 'Esta Kempf', with a stylized, cursive script.

Esta Kempf

Acknowledgement

First and foremost, I would like to thank Prof. Dr. Christian Herta for accepting and reviewing my thesis. My deepest gratitude goes to Dr. Christian Krumnow, whose dedicated supervision and insightful guidance were essential for the completion of this work. His continuous support, including our regular meetings, careful reading of my drafts, and valuable feedback, was incredibly motivating and significantly improved both the quality of this thesis and my academic development.

Zusammenfassung

Deep Learning eröffnet neue Möglichkeiten in der Wettervorhersage, indem es schnelle, datengetriebene Prognosen ermöglicht. Mit GraphCast, einem von DeepMind entwickelten Graph-neuronalen Netzwerk, steht ein Modell zur Verfügung, das insbesondere bei Extremwetterereignissen durch hohe Vorhersagegenauigkeit überzeugt. Aufgrund seines Black-Box-Charakters stellt sich jedoch die Frage nach der physikalischen Nachvollziehbarkeit und Vertrauenswürdigkeit der Modellentscheidungen.

Diese Arbeit untersucht GraphCast-Vorhersagen mithilfe von Methoden der erklärbaren künstlichen Intelligenz (XAI), mit Fokus auf bodennahe Windgeschwindigkeiten während Sturm Zeynep (18. Februar 2022) in Norddeutschland. Dabei wird ein mehrstufiges Analyseverfahren angewendet, das modellagnostische Verfahren, regions- und höhenspezifische Okklusionsanalysen sowie integrierte Gradienten kombiniert, um die physikalische Plausibilität der Vorhersagen zu beurteilen.

Die Analyse zeigt, dass die Vorhersagen von GraphCast maßgeblich durch Windgeschwindigkeit und geopotenzielle Höhe beeinflusst werden, vor allem in meteorologisch relevanten Regionen wie dem Jet Streak über Westeuropa und dem Sturmzentrum. Die Attributionsmuster stimmen weitgehend mit der meteorologischen Theorie überein und deuten darauf hin, dass GraphCast trotz seines datengetriebenen Ansatzes zentrale physikalische Prozesse realitätsnah abbildet.

Über die Fallstudie hinaus entwickelt die Arbeit einen allgemeinen Interpretationsrahmen für Deep-Learning-gestützte Wettermodelle unter realistischen Rahmenbedingungen. Dieser Ansatz verdeutlicht den Mehrwert kombinierter XAI-Methoden zur Öffnung der Black Box komplexer neuronaler Vorhersagesysteme. Die Ergebnisse machen sowohl das Potenzial als auch die Grenzen aktueller Werkzeuge deutlich und unterstreichen die Notwendigkeit einer stärkeren Integration meteorologischer Fachkenntnisse zur Verbesserung von Transparenz und operationeller Verlässlichkeit.

Abstract

Deep learning has opened new frontiers in weather forecasting by enabling fast, data-driven predictions. GraphCast, a graph neural network developed by DeepMind, delivers state-of-the-art accuracy, particularly for extreme weather events. Yet, its black-box nature raises questions about physical interpretability and model trustworthiness.

This thesis examines GraphCast forecasts using explainable AI (XAI) techniques, focusing on near-surface wind predictions during Storm Zeynep (18 February 2022) in northern Germany. A layered attribution strategy is conducted, combining model-agnostic methods, region- and pressure-specific occlusion, and integrated gradients to evaluate the physical plausibility of the model’s internal reasoning.

The results show that GraphCast’s predictions are strongly influenced by physically meaningful variables, particularly wind speed and geopotential height in dynamically important regions such as the Western Europe Jet Streak and the Storm Core. Attribution patterns align well with established atmospheric theory, indicating that GraphCast captures key physical processes despite its data-driven architecture.

Beyond the case study, the thesis proposes a general interpretability framework for deep learning-based weather models under realistic constraints. This approach demonstrates the value of combining complementary XAI methods to open the black box of complex neural forecasting systems. The findings highlight both the promise and the limitations of current tools and point to the need for further integration of meteorological expertise to ensure transparency and operational reliability.

Contents

1. Introduction.....	8
2. State of Art	10
2.1. Weather Forecasting with Deep Learning	10
2.2. Graphical Neuronal Networks and Attention Networks.....	11
2.2.1. Message Passing.....	12
2.2.2. Graph Attention Networks.....	13
2.3. GraphCast Model	15
2.3.1. Encoder.....	16
2.3.2. Processor.....	17
2.3.3. Decoder.....	17
2.3.4. Pretrained Models.....	18
2.4. Explainable AI.....	19
2.5. Model-Agnostic Attribution Methods	19
2.5.1. Feature Occlusion / Input Masking	20
2.5.2. Permutation Feature Importance	20
2.5.3. Shapley Additive Explanations (SHAP).....	21
2.6. Backpropagation-Based Attribution Methods	23
2.6.1. Saliency Maps	23
3. Methods.....	25
3.1. Selection of Weather Event	25
3.2. Data Basis: ERA5 Reanalysis	26
3.3. Climatology-Based Input Masking	27
3.4. Evaluation Metrics	28
3.5. Mask Region Definitions.....	28
3.6. Evaluation Target.....	29
3.7. Feature Occlusion / Input Masking	29
3.8. Permutation Feature Importance	30

3.9. SHAP (KernelSHAP)	30
3.10. Integrated Gradients	31
4. Design	32
4.1. Methodology and Result Artifacts.....	32
4.2. System Architecture	33
4.3. Implementation.....	34
4.3.1. Forecast Package	35
4.3.2. Mask Package	36
4.3.3. Feature Occlusion Package.....	37
4.3.4. Permutation Feature Importance Package	37
4.3.5. SHAP Package.....	38
4.3.6. Saliency Maps Package	38
4.3.7. GCS Utilities Package	38
4.3.8. Plots Package.....	39
4.4. Testing	39
4.5. Privacy and Ethics	40
5. Meteorological Expectations	41
6. Results.....	43
6.1. Analysis of Forecasts.....	43
6.2. Explainability of the Forecast.....	45
6.2.1. Variable Importance via Model-Agnostic Methods.....	45
6.2.2. Feature Occlusion Analysis Across Regions and Pressure Levels	48
6.2.3. Gradients for Spatial Analysis	50
7. Discussion	52
7.1. Analysis of Forecast	52
7.2. Explainability of the Forecast.....	52
7.2.1. Variable Importance via Model-Agnostic Methods.....	53
7.2.2. Feature Occlusion Analysis Across Regions and Pressure Levels	55
7.2.3. Gradients for Spatial Analysis	57
7.2.4. Explaining GraphCast: Challenges and a Structured Interpretation Strategy	59

7.3 Outlook	60
7.3.1. Model Fidelity and Resolution	60
7.3.2. Forecast Horizon	61
7.3.3. Improving Region-Based Occlusion Strategies.....	61
7.3.4. Conditional SHAP and Neural Surrogates	62
7.3.5. Structure-Aware Interpretability	62
7.3.6. Integrating Domain Expertise.....	63
8. Conclusion	64
References	66
Appendix	70
Code Repository	73

1. Introduction

Accurate and timely weather forecasting is essential for modern society, with applications ranging from agriculture and transportation to disaster preparedness and energy management. Extreme weather events such as storms, heatwaves, and floods pose significant threats to infrastructure and human safety, underscoring the need for reliable predictions.

In recent years, data-driven forecasting has emerged as a powerful complement to traditional numerical weather prediction (NWP) systems. By leveraging extensive meteorological reanalysis datasets, deep learning models can learn to emulate atmospheric dynamics directly from historical data, enabling fast and efficient forecasts without explicitly solving physical equations [1].

GraphCast, developed by DeepMind in 2023, represents a major advancement in this domain. Using a Graph Neural Network (GNN) architecture, it models global atmospheric evolution and has demonstrated superior skill compared to conventional forecasting systems across a range of variables and regions, particularly for high-impact weather events [2]. However, despite these promising results, GraphCast remains largely opaque in its internal decision-making. Its complex architecture, high-dimensional inputs, and limited access to internal representations make it difficult to interpret. This poses a key barrier to trust, operational adoption, and integration with meteorological theory, particularly when used for critical scenarios such as extreme weather forecasting.

This thesis addresses this challenge by investigating the physical interpretability of GraphCast forecasts using explainable artificial intelligence (XAI) techniques. The core objective is to analyze how GraphCast arrives at its near-surface wind predictions during Storm Zeynep (18 February 2022), an intense extratropical cyclone that caused widespread damage across northern Germany [3]. The study aims to determine whether the model’s predictions are not only accurate, but also physically plausible and consistent with established atmospheric dynamics.

Given the limited introspection capabilities of the public GraphCast implementation, where direct access to node embeddings, graph attention weights, or edge representations is restricted, many standard GNN explanation methods are not applicable [4]. Instead, this thesis develops and applies a layered interpretability strategy based on model-agnostic and gradient-based techniques. This approach unfolds in three stages: (1) global variable importance analysis using feature occlusion,

permutation feature importance, and KernelSHAP; (2) region- and pressure-specific occlusion focused on meteorological structures such as jet streaks and cyclone cores; and (3) spatial attribution using integrated gradients to visualize the influence of atmospheric patterns at specific forecast locations.

By combining these methods, the thesis not only sheds light on the physical reasoning behind GraphCast’s predictions for Storm Zeynep but also proposes a general framework for interpreting deep learning-based weather models under real-world constraints. This work contributes to the broader goal of making data-driven forecasting systems more transparent, trustworthy, and physically grounded.

2. State of Art

2.1. Weather Forecasting with Deep Learning

Traditional numerical weather prediction (NWP) models have long been the foundation of operational forecasting. These models simulate the physical evolution of the atmosphere by solving complex systems of partial differential equations derived from fundamental physical laws [5]. However, such approaches are computationally expensive. For instance, producing a high-resolution global forecast over ten days can take several hours on a supercomputer with hundreds of processing cores [1], [5]. Furthermore, due to computational limitations, NWP models rely on parameterizations, approximations of unresolved physical processes like convection or cloud formation, which introduce systematic biases and uncertainties [6], [7].

In recent years, data-driven methods based on deep learning have emerged as a promising alternative. Machine learning-based weather prediction (MLWP) frameworks treat forecasting as a supervised learning task: using decades of reanalysis data, neural networks are trained to learn the underlying spatiotemporal dynamics of the atmosphere. While the training phase requires significant computational resources and large-scale GPU or TPU infrastructure, the inference stage is dramatically faster. Once trained, these models can produce ten-day global forecasts in a matter of minutes [2], [8].

Among the early successes in this field is FourCastNet, developed by NVIDIA. It employs Adaptive Fourier Neural Operators to learn spectral representations of global atmospheric fields [8]. It produces forecasts at 0.25° resolution achieving skill comparable to European Centre for Medium-Range Weather Forecasts (ECMWF)'s Integrated Forecast System (IFS) on large-scale variables such as geopotential height and even surpassing it on fine-scale fields like precipitation [8]. Each 24-hour forecast step can be computed in under two seconds on a single GPU [8].

Another prominent contribution is Pangu-Weather, introduced by Huawei. It leverages a 3D Earth-Specific Transformer architecture combined with a hierarchical temporal aggregation strategy. This design enables the model to produce forecasts that outperform even the operational version of ECMWF's IFS while operating on significantly faster time scales [1].

A further advancement in learned weather forecasting is GraphCast, a deep learning-based global weather forecasting model developed by Google DeepMind and released in 2023. Trained on ERA5 reanalysis data from ECMWF, GraphCast is publicly available as a pretrained model. It is based on a Graph Neural Network (GNN) framework that incorporates a Graph Attention Network (GAT) architecture to effectively model spatial dependencies across the globe. The model predicts atmospheric states at time $t + 6$ h, given inputs at t and $t - 6$ h, and can be recursively applied to generate forecasts up to ten days ahead [2]. In empirical evaluations, GraphCast demonstrated strong skill across a variety of meteorological variables. It outperforms the top deterministic operational system ECMWF’s High RESolution forecast (HRES) on 90% of the chosen verification targets and excels at severe event prediction such as tropical cyclones, extreme temperatures or atmospheric rivers [2].

Most recently, DeepMind introduced GenCast in 2024, a diffusion-based generative model that reframes weather forecasting as a probabilistic sequence generation task [9]. At its core, it uses a diffusion model that starts from random noise and progressively refines its outputs through learned denoising steps, ultimately producing physically plausible weather states [9]. In empirical evaluations, it outperformed the ECMWF operational ensemble in over 97% of targets tested and achieved global 15-day forecasts at 0.25° resolution in eight minutes on a single TPU [9].

2.2. Graphical Neuronal Networks and Attention Networks

GraphCast is built upon the GNN paradigm, specifically utilizing a GAT.

GNNs are a class of neural networks designed to operate directly on graph-structured data. A graph is defined as $G = (V, E)$, where V is the set of nodes, and E is the set of edges representing relationships or interactions between the nodes. The structure of the graph is encoded in an adjacency matrix $A \in \{0,1\}^{N \times N}$, where $A_{ij} = 1$ if there is an edge from node i to node j , and $A_{ij} = 0$ otherwise. Additionally, each node is associated with an initial feature vector, and all node features are collected into a matrix $H^{(0)} \in \mathbb{R}^{N \times F}$, where the row $\vec{h}_i^{(0)} \in \mathbb{R}^F$ represents the feature vector of node i [10], [11], [12].

In many advanced GNN architectures, edges can be associated with their own feature vectors and represent properties of the relationship between two nodes [13], [14].

2.2.1. Message Passing

A core mechanism in GNNs is message passing, originally developed in molecular graph modeling [13]. Hereby, each node transmits information (messages) to its neighboring nodes based on its current state. Simultaneously, it receives messages from its neighbors, which are subsequently aggregated using a predefined or learned function. This aggregated information is then used to update the node’s own state, enabling it to incorporate contextual information from its local graph neighborhood. This process is typically formalized as:

$$\vec{h}_i^{(l+1)} = \text{UPDATE}^{(l)} \left(\vec{h}_i^{(l)}, \text{AGGREGATE}^{(l)} \left(\left\{ \vec{h}_j^{(l)} : j \in N(i) \right\} \right) \right)$$

where $\vec{h}_i^{(l)}$ is the feature of node i at layer l , $N(i)$ is the set of neighbors of node i with $N(i) = \{j \mid A_{ij} = 1\}$ and AGGREGATE and UPDATE are learnable functions [11]. This process is illustrated in Figure 1.

When edge features e_{ij} are present, the message passing mechanism is extended to take them into account:

$$\vec{h}_i^{(l+1)} = \text{UPDATE}^{(l)} \left(\vec{h}_i^{(l)}, \text{AGGREGATE}^{(l)} \left(\left\{ \text{MSG}(\vec{h}_i^{(l)}, \vec{h}_j^{(l)}, \vec{e}_{ij}^{(l)}) : j \in N(i) \right\} \right) \right)$$

Here, MSG is a learnable function (often a multi-layer perceptron (MLP)) that generates the message from node j to node i , incorporating both node and edge features. This is the formulation used in GraphCast, where messages depend not only on node embeddings but also on geometric relationships captured by edge features [13], [14].

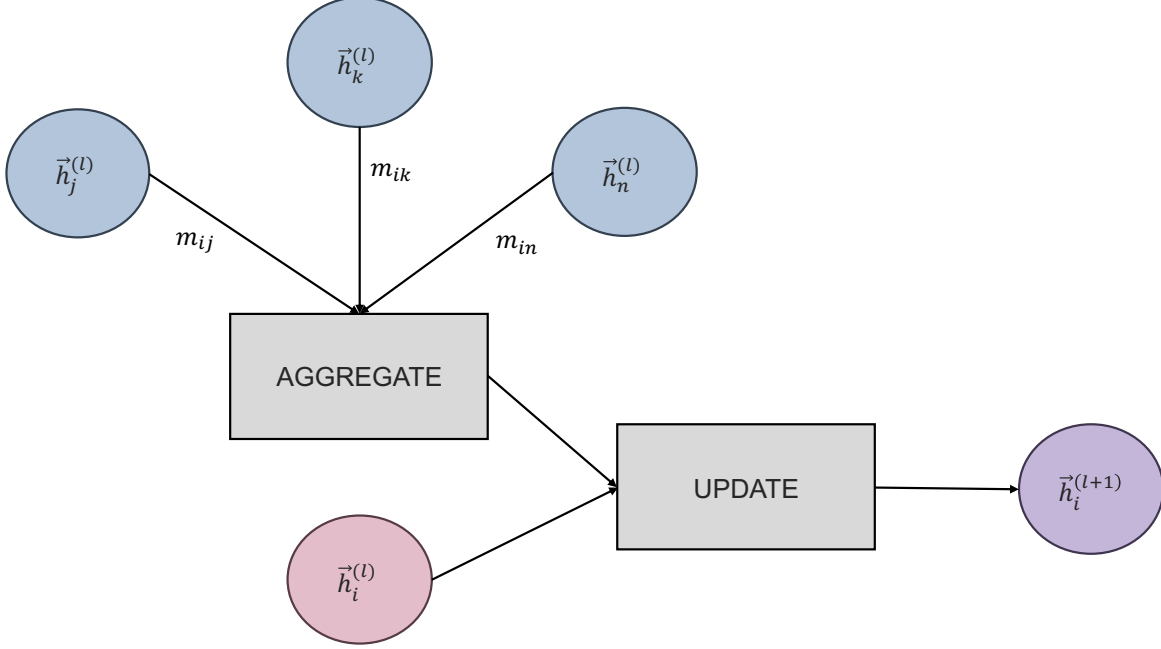


Figure 1: Illustration of the message-passing mechanism in a GNN.

Neighbor node states $\vec{h}_j^{(l)}$ are transformed into messages m_{ij} , aggregated via a learnable function, and combined with the node's current state $\vec{h}_i^{(l)}$ through an UPDATE function to produce the updated representation of the next layer $\vec{h}_i^{(l+1)}$.

2.2.2. Graph Attention Networks

GATs extend the traditional GNN architecture by introducing a learnable attention mechanism that quantifies the relative importance of neighboring nodes during the message-passing process.

The process begins with a linear transformation of each node's feature vector. Given a node i with feature vector $\vec{h}_i \in \mathbb{R}^F$, a shared learnable weight matrix $W \in \mathbb{R}^{F' \times F}$ is applied to project the features into a latent space:

$$\vec{z}_i = W\vec{h}_i$$

Here, $\vec{z}_i \in \mathbb{R}^{F'}$ is the transformed feature vector of node i [11], [12].

Next, for each pair of connected nodes i and j , an attention score s_{ij} is computed using a shared learnable attention vector $\vec{a} \in \mathbb{R}^{2F'}$. This score measures the importance of node j 's features to node i , and is calculated as follows:

$$s_{ij} = \text{LeakyReLU}(\vec{a}^T [\vec{z}_i || \vec{z}_j])$$

where $[\vec{z}_i || \vec{z}_j]$ denotes the concatenation of the two transformed feature vectors, and LeakyReLU is applied as a non-linear activation function to allow for small negative values [11], [12].

To ensure comparability and numerical stability, these attention scores can be normalized using the softmax function across all neighbors $N(i)$ of node i :

$$\alpha_{ij} = \text{softmax}(s_{ij}) = \frac{\exp(s_{ij})}{\sum_{k \in N(i)} \exp(s_{ik})}$$

With this, each neighbor j has a weight $\alpha_{ij} \in (0,1)$ and $\sum_j \alpha_{ij} = 1$.

Finally, the new node representation $\vec{h}_i^{(l+1)}$ is computed as a weighted sum of the transformed features from its neighbors:

$$\vec{h}_i^{(l+1)} = \sigma \left(\sum_{j \in N(i)} \alpha_{ij} \vec{z}_j \right)$$

Here, σ is a non-linear activation function such as ReLU or ELU, and l denotes the layer index in a multi-layer architecture [11], [12].

Practical implementations and extensions of GATs – including GraphCast – can incorporate edge features by modifying the attention mechanism to include edge embeddings. For instance, attention weights can be computed from concatenated node and edge features, enabling the model to leverage spatial or relational context more effectively. In this extended formulation, the attention score is computed as follows, incorporating both node and edge features [2] [13]:

$$s_{ij} = \text{LeakyReLU}(\vec{a}^T [\vec{z}_i || \vec{z}_j || \vec{e}_{ij}])$$

2.3. GraphCast Model

GraphCast is built around an encode-process-decode GNN framework that combines structured gridded inputs with a learned mesh-based representation. At each forecast step, it takes two consecutive weather states and produces the next predicted state by applying a forward pass over its hybrid graph structure, formally represented as:

$$\hat{X}^{t+1} = \text{GraphCast}(X^t, X^{t-1})$$

The model operates on a hybrid graph that integrates both a regular latitude-longitude grid and a multi-resolution icosahedral mesh. This design enables efficient long-range spatial communication across the globe while preserving the physical coherence required for atmospheric forecasting [2].

The computational graph used by GraphCast is formally defined as $G = (V^G, V^M, E^M, E^{G2M}, E^{M2G})$ where V^G and V^M are sets of grid and mesh nodes, respectively. The edges E^M , E^{G2M} , and E^{M2G} describe the different edges in the graph, whereas E^M are the bidirectional edges between the mesh nodes and E^{G2M} , E^{M2G} are unidirectional edges representing communication from grid to mesh, and mesh to grid [2].

The set V^G contains 1,038,240 grid nodes, each corresponding to a vertical atmospheric column at a fixed location on a 0.25° latitude–longitude grid. Each grid node v_i^G is associated with a feature vector:

$$v_i^{G, \text{features}} = [x_i^{t-1}, x_i^t, f_i^{t-1}, f_i^t, f_i^{t+1}, c_i]$$

where x_i^t represents the full atmospheric state at time t for node i , including all five surface variables and atmospheric variables across 37 pressure levels (six variables per level) (variables and levels are listed in Table A1, Appendix). The terms f_i^t are time-dependent forcing terms (e.g., solar radiation and temporal encodings), and c_i denotes static geographic features such as the land-sea mask and trigonometric functions of the location. Each node thus encodes a total of 474 input features [2].

The set V^M comprises the mesh nodes, which are positioned on a refined icosahedral mesh that has been subdivided six times, resulting in 40,962 nodes at the finest resolution. Unlike the latitude-longitude grid, the mesh provides a near-uniform spatial distribution of nodes over the globe and

avoids the singularities found at the poles. This mesh structure is organized hierarchically, from coarse (M^0) to fine (M^6) resolutions, where nodes at coarser levels are strict subsets of finer ones. Each mesh node v_i^M includes positional encodings in the form of trigonometric functions of latitude and longitude [2].

Edges in E^M are bidirectional and span all levels of the mesh hierarchy, from M^0 to M^6 . While the mesh is conceptually organized into multiple levels, GraphCast operates on a single unified mesh at the finest resolution (M^6), while the coarser levels (M^0 to M^5) provide hub nodes for efficient long-range communication. This hierarchical edge structure, known as the multi-mesh, enables both short- and long-range message passing in a spatially efficient manner [2].

Each edge is associated with a 4-dimensional geometric feature vector, encoding the length and relative direction between connected nodes in a locally rotated spherical coordinate frame. The grid-to-mesh edges E^{G2M} and mesh-to-grid edges E^{M2G} are also assigned edge features based on similar geometric principles [2].

GraphCast's architecture consists of three major components: an encoder, a processor, and a decoder [2].

2.3.1. Encoder

The encoder is responsible for mapping the structured grid-based input into latent representations on the mesh. It begins by embedding the features of all node and edge types into a shared latent space using five distinct MLPs, one each for each graph component V^G, V^M, E^M, E^{G2M} and E^{M2G} . Following this, a single message-passing step is executed over the grid-to-mesh subgraph $G^{G2M} = (V^G, V^M, E^{G2M})$. This message passing is performed using an extended interaction network that first updates edge representations based on their source and target nodes and then aggregates these updated messages at each mesh node. Grid nodes are updated individually via MLPs (without aggregation), as they are not receivers in this step. After this update, residual connections are applied to all updated nodes and edges, enabling more stable learning. This process is visualized in Figure 2a [2].

2.3.2. Processor

Once encoded, the latent representations are passed to the processor, which is implemented as a deep 16-layer GNN operating exclusively on the mesh subgraph $G^M = (V^M, E^M)$. At each layer, the model updates mesh edges based on their adjacent node features, then updates each node by aggregating messages from incoming edges. This iterative message passing is again performed using MLPs, with separate weights for each layer. Critically, the multi-mesh edge design ensures that information can propagate both locally and globally across the planet, enabling the model to learn fine-grained physical dynamics as well as long-range interactions. Residual connections are used at every layer to preserve signal integrity over depth. Figure 2b illustrates the mesh-centric processing structure, while Figure 2d shows the mesh hierarchy that underlies its connectivity [2].

2.3.3. Decoder

The final stage is the decoder, which projects the mesh-based latent state back into the gridded format required for forecasting. A message-passing step is performed on the mesh-to-grid subgraph $G^{M2G} = (V^G, V^M, E^{M2G})$, where each mesh-to-grid edge is first updated using the current node and edge states, followed by an aggregation step at each grid node. Unlike in the encoder, only grid nodes are updated in this step, as the mesh representation is no longer needed after the decoding stage. A residual connection is again applied to each grid node’s updated embedding. Finally, an MLP maps the updated grid node states to the target weather variables. This prediction is added to the previous state X^t , forming the model’s final output $\hat{X}^{t+1} = X^t + \hat{Y}^t$, as visualized in Figure 2c [2].

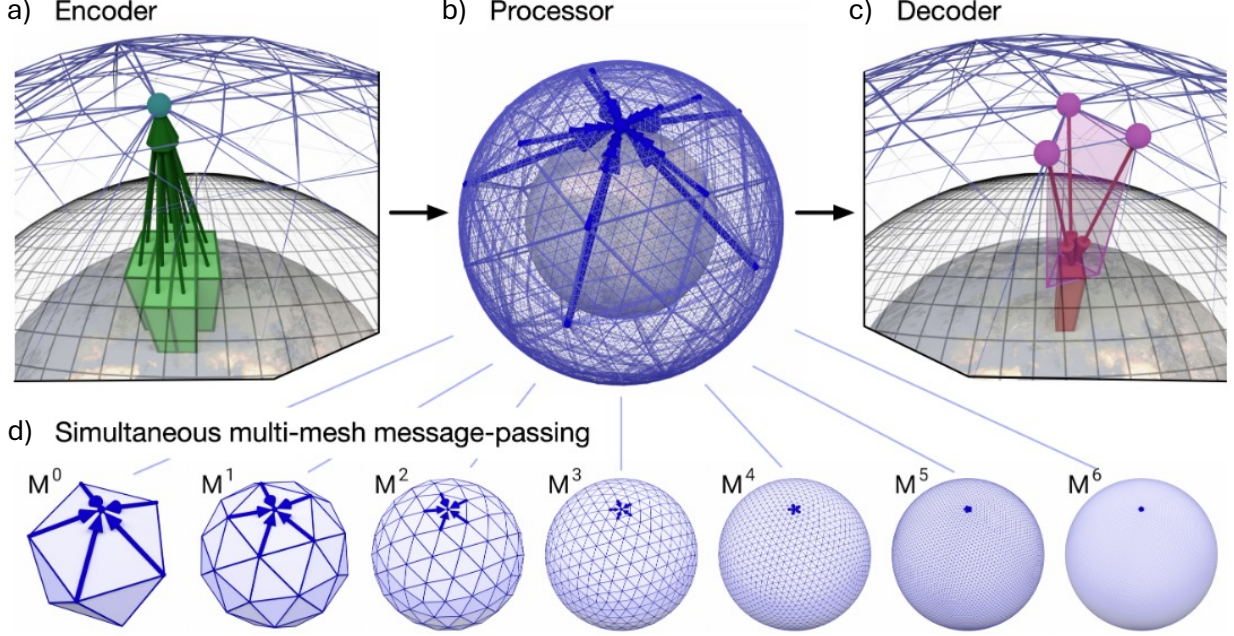


Figure 2: Overview of the GraphCast architecture

(a) The encoder maps structured atmospheric input fields into latent graph node embeddings using grid-to-mesh projections. (b) The processor performs 16 layers of message passing on a multi-resolution icosahedral mesh. (c) The decoder projects mesh-based representations back to grid-based forecasts via mesh-to-grid edges. (d) The mesh hierarchy from M^0 to M^6 is used to construct a unified, fine-resolution mesh with multi-scale connectivity. Nodes from coarser levels are subsets of finer ones, enabling local and long-range communication through hierarchical edges. Adapted from [2].

2.3.4. Pretrained Models

To support efficient experimentation and deployment, three pretrained variants of GraphCast are publicly available. The original GraphCast model, as introduced in the corresponding publication, operates at a spatial resolution of 0.25° with 37 pressure levels. It was trained on ERA5 reanalysis data from 1979 to 2017 and is designed for high-resolution, high-skill forecasting. For environments with limited computational resources, a smaller variant – GraphCast_small – is provided. This version runs at 1.0° resolution with only 13 pressure levels and a simplified graph mesh, making it significantly less demanding in terms of memory and compute. Lastly, the GraphCast_operational model combines high spatial resolution (0.25°) with a reduced number of pressure levels (13). It is pretrained on ERA5 (1979–2017) and fine-tuned on ECMWF HRES forecast data (2016–2021), enabling operational usage where precipitation inputs are unavailable. All models are accessible through the official GraphCast GitHub repository maintained by DeepMind [4].

2.4. Explainable AI

As machine learning models become more powerful and complex, understanding how they make predictions becomes increasingly important. Explainable AI (XAI) refers to a set of methods and techniques that aim to make the decision-making processes of machine learning models more transparent and interpretable.

XAI techniques can be global, providing insights into the overall behavior of the model across an entire dataset, or local, focusing on individual predictions. Moreover, XAI methods are commonly divided into two categories: model-specific approaches that rely on access to the internal model structure and model-agnostic approaches, which treat the model as a black box and rely solely on its input-output behavior [15], [16].

Given the restricted access to GraphCast’s internal representations, this thesis primarily employs model-agnostic methods and gradient-based saliency techniques. These approaches are well-suited to explain predictions even when the underlying architecture is not fully transparent, enabling both global (feature importance) and local (spatial attribution) analyses.

2.5. Model-Agnostic Attribution Methods

Model-agnostic attribution methods aim to explain model predictions without requiring access to internal parameters or architectures. These methods typically work by perturbing input features and observing the resulting changes in the model output, thereby estimating each feature’s contribution. Such methods are particularly attractive because they can be applied uniformly across different models and data types, regardless of internal complexity [14].

Prominent examples include feature occlusion, pfi, and KernelSHAP, which are widely used for evaluating the global influence of input variables on predictive models.

2.5.1. Feature Occlusion / Input Masking

Feature occlusion, also known as input masking, originated in the field of computer vision and involves systematically masking individual input variables and measuring the resulting change in the model’s output [17]. The greater the change caused by masking, the more relevant the occluded feature is considered for the prediction. Formally, the importance of feature i can be expressed as:

$$I_i = \mathcal{L}(f(x), f(occ_i(x))),$$

where f is the model, $f(x)$ the original input, $occ_i(x)$ the masked input, and $\mathcal{L}$ a function measuring the difference between predictions (e.g., absolute difference).

Masking can be implemented in various ways, such as zeroing out values, replacing them with an appropriate mean, adding noise, or applying dropout.

2.5.2. Permutation Feature Importance

Permutation feature importance (pfi) is a model-agnostic method for quantifying the importance of input features by evaluating the impact of random perturbations on model performance. The key idea is that if a feature is important for prediction, disrupting its structure should lead to a measurable decrease in forecast skill [18], [19].

The procedure begins by computing a baseline performance metric (e.g., RMSE or CRPS) using the model’s predictions on an unaltered test set. Then, for each input feature individually, the values are randomly permuted across their original domain – typically in space, time, or both – while keeping all other inputs unchanged. This operation preserves the original feature distribution but breaks any spatial or temporal correlation it has with the target variable. The model is then re-evaluated on the permuted inputs, and the performance degradation relative to the baseline is recorded as the feature’s importance score [18], [19].

Formally, let f be the trained model, X the unaltered input, and $X_{perm}^{(j)}$ the input with feature j permuted. Then, the importance of feature j can be expressed as:

$$PFI_j = Error\left(f\left(X_{perm}^{(j)}\right)\right) - Error\left(f(X)\right).$$

A larger value of PFI_j indicates a greater drop in performance and hence higher feature relevance. Negative values can occur when permutation accidentally improves performance, which typically suggests that the feature adds noise or has low predictive value [18], [19]. To account for the randomness introduced by shuffling, the permutation process is often repeated multiple times, and the mean pfi along with the standard error of the mean is reported to assess the stability and significance of the importance scores.

2.5.3. Shapley Additive Explanations (SHAP)

Shapley values originated from cooperative game theory, aiming to fairly distribute a reward among a set of players M based on their marginal contributions. A value function v assigns each subset (coalition) $S \subseteq M$ a payout $v(S)$. The Shapley value ϕ_i for player i is defined as the average marginal contribution of i to every possible coalition:

$$\phi_i = \sum_{S \subseteq M/\{i\}} \frac{|S|! (|M| - |S| - 1)!}{|M|!} [v(S \cup \{i\}) - v(S)]$$

This formula satisfies four desirable axioms: efficiency (all contributions sum to the total gain), symmetry (equally contributing players receive equal value), the dummy rule (non-contributing players receive zero), and additivity (values over combined games add) [19], [20], [21].

In the context of a machine learning model f , SHapley Additive exPlanations (SHAP) interprets each feature as a player and the model's output $f(x)$ as the reward. The feature attribution $\phi_i(x)$ is defined analogously by evaluating the effect of including feature i in different subsets $S \subseteq M \setminus \{i\}$, where only the subset S is visible:

$$\phi_i(x) = \sum_{S \subseteq M/\{i\}} \frac{|S|! (|M| - |S| - 1)!}{|M|!} [f_{S \cup \{i\}}(x_{S \cup \{i\}}) - f_S(x_S)]$$

Here, $f_S(x_S)$ denotes the model's expected output conditioned on only the features in subset S [22]. This definition preserves the Shapley axioms and satisfies additional properties – local accuracy (the sum of feature attributions equals the model output), missingness (features absent in the

coalition receive zero attribution), and consistency (if a feature's influence grows in a new model, its attribution cannot decrease) [21].

Computing this exactly requires evaluating f on all $2^{|M|}$ subsets, which becomes infeasible as the number of features grows. To address this, SHAP introduces a unified explanation model in which any explanation can be written as a linear function over binary feature presence vectors $z' \in \{0,1\}^{|M|}$:

$$g(z') = \phi_0 + \sum_{i=1}^{|M|} \phi_i z'_i$$

where $z'_i = 1$ indicates feature i is present and $z'_i = 0$ that it is not [21].

To efficiently approximate Shapley values, KernelSHAP was introduced as a model-agnostic algorithm that sidesteps full enumeration of all feature subsets. Instead, KernelSHAP samples a manageable number of coalitions z' , evaluates the model on masked versions of the input $x_{z'}$, and fits a weighted linear regression to estimate the attributions ϕ_i . The regression is weighted using the Shapley kernel:

$$\pi(z') = \frac{(|M| - 1)}{\binom{|M|}{|z'|} |z'| (|M| - |z'|)}$$

which reflects the combinatorial importance of each coalition size. The resulting coefficients approximate the true Shapley values using only a modest number of model evaluations [19] [22].

2.6. Backpropagation-Based Attribution Methods

Backpropagation-based attribution methods explain model predictions by computing the gradients of the model output with respect to its input features. These gradients indicate how small perturbations of the input would influence the prediction, making them powerful tools for identifying which features are most relevant for a given forecast. Such methods rely on the model being differentiable and require access to its computational graph to perform automatic differentiation [23].

Although often classified as white-box techniques, in practice they only require gradient access rather than full transparency of the model parameters or architecture. For models like GraphCast, implemented in JAX and Haiku, gradient computation with respect to input fields is possible. This makes saliency maps particularly suitable for analyzing local feature attributions.

2.6.1. Saliency Maps

Saliency maps are a class of explanation techniques that visualize which input features most influence a model’s prediction. Originally introduced in the context of image classification, they rely on backpropagation to compute local sensitivity of the model output with respect to its input [19], [24], [25].

The simplest form, often referred to as a vanilla saliency map, is based on the gradient of the output score with respect to the input features. Let $\mathbb{R}^n \rightarrow \mathbb{R}$ be a differentiable model and $x \in \mathbb{R}^n$ an input vector. The saliency score $S_i(x)$ for the i -th input feature is defined as: $S_i(x) = \left| \frac{\partial f(x)}{\partial x_i} \right|$.

This gradient quantifies how a small perturbation in input dimension x_i affects the output $f(x)$. A higher absolute value indicates greater local influence of feature i on the prediction. The full saliency map is thus the vector $\nabla_x f(x)$ visualized as a heatmap over the input dimensions [19].

However, vanilla saliency maps suffer from gradient saturation: when the model’s prediction function is locally flat, the gradient can be zero even if the feature strongly influences the output. This often leads to attributions that are incomplete or misleading, particularly in deep ReLU networks where activations can saturate [19], [26].

A simple extension to raw gradients is the gradient $\times$ input method, which rescales each sensitivity by the corresponding feature value:

$$S_i(x) = x_i \frac{\partial f(x)}{\partial x_i}$$

This approach emphasizes features that are both present (i.e., have non-zero values) and influential according to the gradient. While it does not resolve saturation issues, it often produces more interpretable attributions by down-weighting inactive features and amplifying the contribution of features that actively shape the prediction [27]. Positive scores denote features that support the prediction; negative scores highlight features that oppose it.

A more principled alternative is integrated gradients. Integrated gradients average gradients along a straight-line path from a baseline x' to the input x :

$$IG_i(x) = (x_i - x'_i) \int_{\alpha=0}^1 \frac{\partial f(x' + \alpha(x - x'))}{\partial x_i} d\alpha$$

By integrating over this path, integrated gradients recover gradients from non-saturated regions, avoiding the zero-gradient issue. Furthermore, it satisfies desirable axioms such as sensitivity, meaning that features that cause a change in prediction between x' and x always receive non-zero attribution, and implementation invariance, ensuring that attributions depend only on the function represented by the network, not its architecture [24], [27]. A positive value means the feature enhances the predicted output relative to the baseline, whereas a negative value suppresses it.

3. Methods

3.1. Selection of Weather Event

To evaluate the predictive performance and interpretability of the GraphCast model under extreme conditions, the extratropical cyclone Zeynep (internationally referred to as Eunice) was selected as a case study. Zeynep developed rapidly along the polar front over the north Atlantic on February 16, 2022, driven by a highly dynamic jet stream and warm, moist subtropical air. The storm underwent explosive cyclogenesis, with its central pressure dropping by over 30 hPa in less than 24 hours – a hallmark of rapid intensification. Within three days, Zeynep traveled approximately 4,000 kilometers, impacting large areas of Western and Central Europe. In Germany, the storm's strongest winds were recorded in the northern regions, particularly along the North Sea coast. Gusts frequently exceeded 120 km/h and reached up to 162 km/h at the Alte Weser lighthouse, making it one of the most significant storm events in the region in recent years. While Zeynep did not surpass historical records across all stations, it brought widespread damage and disruption, especially in coastal and lowland areas of northern Germany [3].

For this study, the 10 m wind speed is used as the primary variable to evaluate the severity and dynamics of the storm. Wind at the Earth's surface is typically described using two orthogonal components: the *u*-component, representing the wind blowing from west to east (zonal direction), and the *v*-component, representing the wind blowing from south to north (meridional direction). These components are combined to compute the scalar wind speed magnitude using the Pythagorean theorem:

$$wind\ speed = \sqrt{u^2 + v^2}$$

To assess the temporal evolution of the storm's intensity, Figure 3 shows the mean 10 m wind speed over northern Germany on February 18, 2022, based on ERA5 reanalysis data. A clear intensification is observed during the evening hours, with the peak wind speeds occurring at 22:00 UTC, coinciding with the core phase of storm Zeynep's impact in the region. Accordingly, all subsequent predictive and interpretability analyses focus on the 22:00 UTC time slice.

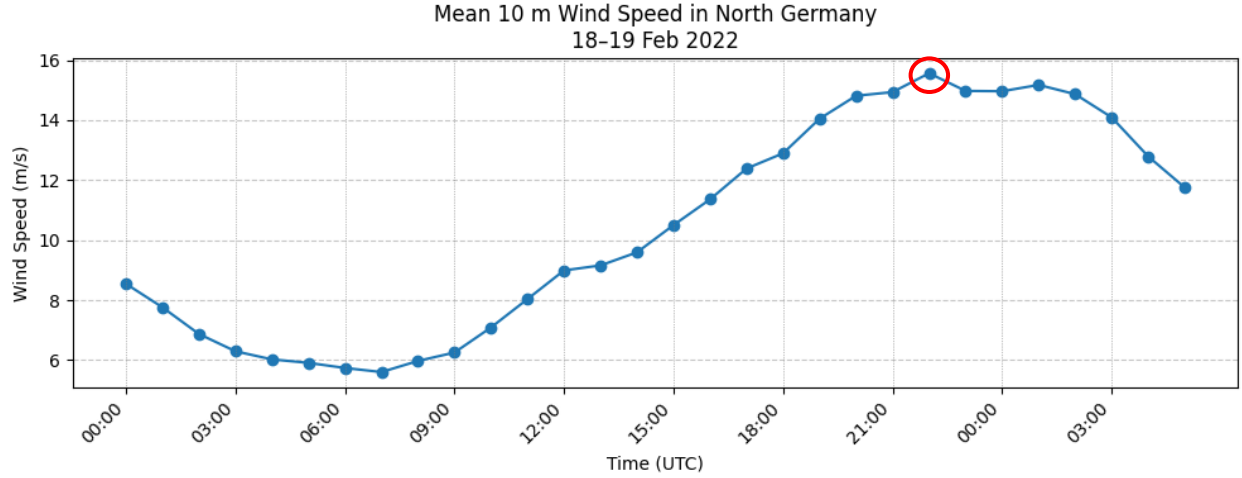


Figure 3: Mean 10 m wind speed over northern Germany on February 18-19, 2022, derived from ERA5 reanalysis data

The average was computed over a spatial domain bounded by latitudes 53°N–55°N and longitudes 8°E–10°E. A maximum is reached at 22:00 UTC (red circle), the time chosen for detailed GraphCast evaluation.

3.2. Data Basis: ERA5 Reanalysis

The analysis conducted in this study is based on meteorological data from ERA5, the fifth-generation atmospheric reanalysis produced by the ECMWF as part of the Copernicus Climate Change Service. ERA5 provides a globally consistent dataset by combining model simulations with a vast array of historical observations through advanced data assimilation techniques based on physical laws. It offers hourly data on a regular $0.25^\circ \times 0.25^\circ$ latitude-longitude grid, with global coverage extending from 1940 to the present [28].

In the context of this work, ERA5 serves two primary purposes: the training and input basis for the GraphCast deep learning forecasting models, and a reference for evaluating model predictions during case study analyses.

All data were retrieved using the official Climate Data Store API (CDS API) and stored in NetCDF format. The processing and analysis were conducted using the Python library xarray, which allows efficient handling of large multidimensional climate datasets.

3.3. Climatology-Based Input Masking

A climatology mask provides the neutral baseline. For every ERA5 variable the 37-year mean (1979-2015) conditioned on day-of-year and UTC hour was computed:

$$Mask(var, d, h) = \frac{1}{37} \sum_{y=1979}^{2015} ERA5_y(var, d, h)$$

Replacing the storm-time value by this mask removes event-specific signals while retaining the correct seasonal and diurnal structure. The mask is applied elementwise so that all downstream XAI tests share a single, meteorologically plausible reference state.

In this study, masks were required for the timestamps that coincide with the evaluation windows of storm Zeynep, 18th of February 2022 at 10:00, 16:00 and 22:00 UTC. The 10 m wind speed by GraphCast during the peak of storm Zeynep (22:00 UTC) and the corresponding climatological baseline are compared in Figure 4. The contrast highlights the storm’s anomalously high wind speeds relative to the seasonal average.

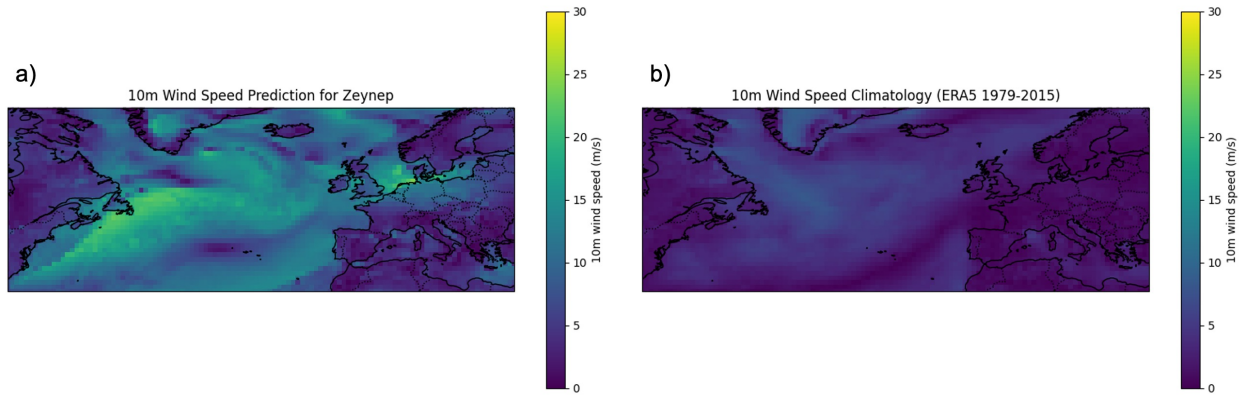


Figure 4: Comparison 10 m wind speed ERA5 data on the 18th of February 2018 vs. average 1979-2015

(a) 10 m wind speed forecast from GraphCast for storm Zeynep at 22:00 UTC on February 18, 2022. (b) Climatological 10 m wind speed from ERA5, averaged over 1981–2020 for the same day and hour (February 18, 22:00 UTC). The climatology serves as a baseline for input masking in feature attribution experiments.

3.4. Evaluation Metrics

To quantify model performance, the Mean Absolute Error (MAE) and Root Mean Square Error (RMSE) were computed over the forecast region. Let y_i be the observed value and $\hat{y}_i$ the predicted value at each grid point i , for a total of N points. The metrics are defined as:

$$MAE = \frac{1}{N} \sum_{i=1}^N |\hat{y}_i - y_i|, \quad RMSE = \sqrt{\frac{1}{N} \sum_{i=1}^N (\hat{y}_i - y_i)^2}$$

MAE measures the average absolute error, while RMSE gives higher weight to larger deviations.

3.5. Mask Region Definitions

Spatial regions used for XAI analyses were defined based on their meteorological significance, as outlined in section 5. Meteorological Expectations. For implementation purposes, the following bounding boxes were applied:

- Atlantic Jet Stream: 40°N–60°N, 80°W–20°W
- Western Europe Jet Streak: 45°N–60°N, 20°W–11°E where wind speed at 300 hPa exceeds 70 m/s
- Low-Pressure North: 60°N–70°N, 20°W–30°E
- Storm Core (10:00 UTC): 54°N–58°N, 7.5°W–1°W
- Storm Core (16:00 UTC): 55°N–58°N, 2°W–8°E

The Atlantic Jet Stream and Western Europe Jet Streak were identified via the wind speed field at 300 hPa (Figure A1, Appendix). The Storm Core locations and Low-Pressure North were determined using the geopotential height field at 850 hPa (Figure A1, Appendix).

3.6. Evaluation Target

Forecasts were generated in a single-step configuration, using input fields at 10:00 and 16:00 UTC on 18 February 2022 to predict the target output at 22:00 UTC the same day.

For feature occlusion, pfi, and SHAP the evaluation target is the predicted surface wind speed over northern Germany, spatially averaged over the region 53-55°N, 8-10°E. This area was selected to capture the broader wind impact zone of storm Zeynep over land.

In contrast, the integrated gradients method requires a scalar output and thus operates on a single grid point. For this purpose, the location 54.0°N, 9.0°E was selected, as it recorded the highest surface wind speed on land during Zeynep's passage.

The forecasts are based on the GraphCast_small configuration, which was selected for all experiments due to its significantly lower computational cost while maintaining comparable performance for mid-latitude storms such as Zeynep. This choice enables a comprehensive evaluation of attribution methods without being limited by resource constraints.

3.7. Feature Occlusion / Input Masking

For each feature or feature combination of interest, the corresponding values in the model input are replaced with a climatological baseline, while all other input fields remain unchanged. GraphCast is then run with this perturbed input to produce a forecast with a 6 h lead time. The resulting forecast is compared to the unperturbed (reference) forecast, and the difference is recorded as a measure of feature importance.

3.8. Permutation Feature Importance

For each candidate feature, a binary mask is defined to specify the grid points where values are permuted. Permutations are applied independently for each variable and pressure level. Five repetitions are performed per feature to account for randomness introduced by shuffling. The original forecast produced from the unmodified input serves as the baseline. Feature importance is quantified using the MAE and RMSE between perturbed and baseline forecasts. Variability in the results is assessed via the standard error of the mean.

3.9. SHAP (KernelSHAP)

Shapley values were computed using the model-agnostic KernelSHAP algorithm and implemented via the shap Python library [22]. Graph-specific approaches such as GraphSHAP and SubgraphX are not applicable as these methods assume a single, explicitly defined graph as input, typically with dozens to a few hundred nodes, and require access to the model’s internal graph representation or node embeddings. In contrast, GraphCast operates on global gridded fields represented as dense tensors and constructs its internal graph mesh implicitly during inference. Neither the input format nor the architecture exposes a manipulable graph structure compatible with graph-based explainability techniques. Moreover, scaling such methods to $\sim 40\,000$ spatial nodes would be computationally infeasible.

KernelSHAP circumvents these limitations by treating the model as a black box. A set of k input features, each defined as a variable or variable group, is used to construct binary coalition vectors $z \in \{0,1\}^k$. For each coalition, a perturbed input is generated by masking excluded features with a climatological baseline (see chapter 3.3. Climatology-Based Input Masking) and retaining only the selected ones. The background sample consists of a fully masked input ($z = 0$), and the evaluation point corresponds to the fully unmasked input ($z = 1$).

3.10. Integrated Gradients

Integrated gradients compute feature attributions by integrating the gradient of the model output with respect to each input feature along a straight-line path from a baseline input to the actual input. In this case, the baseline was a climatological average. The path integral is approximated by a Riemann sum using $m = 20$ steps:

$$IG_i(x) = (x_i - x'_i) \frac{1}{m} \sum_{k=1}^m \frac{\partial f \left(x' + \frac{k}{m} (x - x') \right)}{\partial x_i}$$

This yields a saliency map indicating the relative contribution of each input field to the wind speed prediction at the target location.

4. Design

4.1. Methodology and Result Artifacts

The system developed for this thesis follows a modular, reproducible research workflow that integrates data handling, model inference, and explainability. The core logic is encapsulated in a custom Python package composed of independently testable submodules. These handle tasks such as cloud storage access, input preparation for the GraphCast model, forecast generation, climatology-based masking, and the execution of attribution methods including feature occlusion, pfi, KernelSHAP, and saliency maps.

The codebase is designed for compatibility with both local and cloud-based infrastructure. Compute tasks can be executed on Google Cloud Platform using dynamically provisioned TPUs (see section 4.2. System Architecture). Intermediate and final results are automatically saved locally and optionally synchronized to cloud storage, enabling reproducible and scalable experimentation.

All attribution experiments are orchestrated using a suite of structured Jupyter notebooks, which serve as entry points to the package functionality.

The scientific outputs generated during the analysis include perturbed forecast visualizations, heatmaps of pressure-level and region-specific feature occlusion across key variables and saliency maps. All outputs were produced in a structured, script-driven manner, allowing for full traceability and reproducibility of the experiments.

The complete implementation of the described system, including all packages and analysis scripts, is available in the GitHub repository listed in Appendix – Code Repository.

4.2. System Architecture

The system architecture enables seamless operation in both local and cloud environments.

Cloud infrastructure is defined programmatically using Pulumi, an infrastructure-as-code framework, with the deployment logic implemented in Go. Depending on the active stack configuration, Pulumi provisions TPU v5-LitePod compute resources and/or a Google Cloud Storage (GCS) bucket. Upon creation, the cloud environment is automatically bootstrapped with setup scripts, which are uploaded and executed in a predefined sequence. This is followed by the installation of dependencies and deployment of the custom Python package and environment configuration. The GCS bucket is automatically mounted to the TPU for persistent storage access (Figure 5).

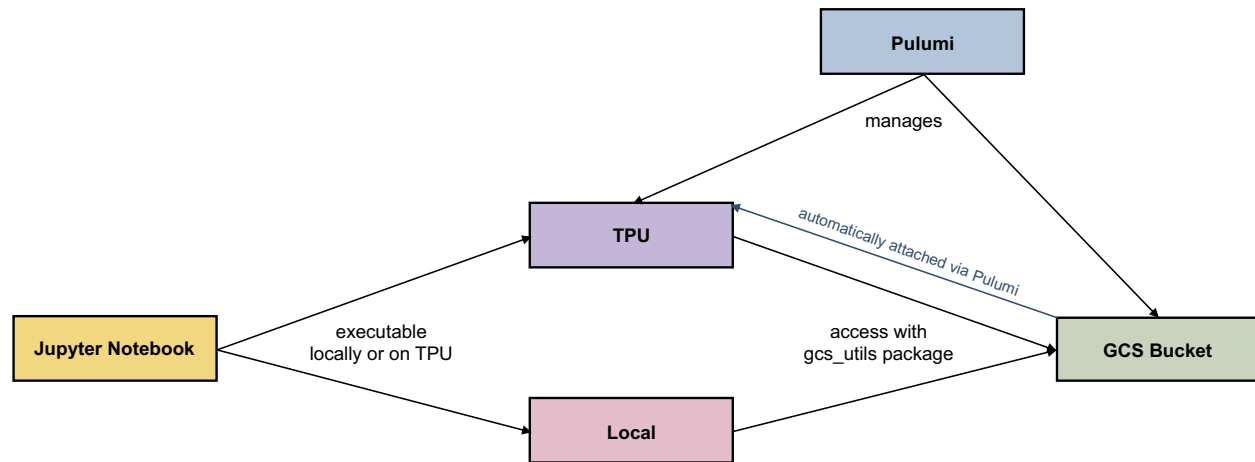


Figure 5: System architecture for hybrid local-cloud operation

Jupyter notebooks serve as the primary interface to orchestrate workflows, which can be executed either locally or on cloud-based TPU instances. Infrastructure provisioning is fully automated using Pulumi, which dynamically configures both compute resources and a GCS bucket. The GCS bucket functions as a centralized data exchange layer accessible from both local and remote environments. Its integration with the TPU is handled automatically through Pulumi, while read/write access is managed via the `gcs_utils` module within the codebase.

The system is stateless by design: cloud compute instances are ephemeral and re-instantiated from scratch for each run. This ensures consistent execution environments, eliminates configuration drift, and promotes clean experimentation workflows.

To support both local and remote execution without changes to the codebase, a shared data interface is implemented via GCS. A dedicated storage bucket is provisioned during deployment and configured with the appropriate IAM permissions. Within the codebase, I/O routines accept an `upload_to_gcs` flag, allowing outputs to be synchronized to the cloud in parallel with local storage. This mechanism ensures intermediate and final results are accessible and persist independently of the lifecycle of any particular instance.

Configuration parameters such as GCP project ID, SSH credentials, and the target bucket name are specified via a `.env` file. Pulumi stacks define resource-specific configurations for TPU and storage. If available, Copernicus CDS API credentials are also uploaded to the server to enable cloud-side reanalysis data access.

This hybrid local-cloud architecture provides a consistent and unified execution model, with full automation of resource provisioning, environment setup, and result synchronization. It ensures high reproducibility, scalability, and minimal manual intervention.

4.3. Implementation

The implementation is organized into modular Python packages, each targeting a specific component of the forecasting and attribution pipeline. This structure ensures clean separation of concerns, supports reproducibility, and enables scaling across experiments. The components include data preparation, model inference, masking, attribution methods (feature occlusion, pfi, SHAP, saliency), and visualization tools. The attribution pipeline of the packages is shown in Figure 6. All components are implemented to support execution in both local and remote environments. Where output is written to disk, an optional `upload_to_gcs` flag enables automatic synchronization with a GCS bucket.

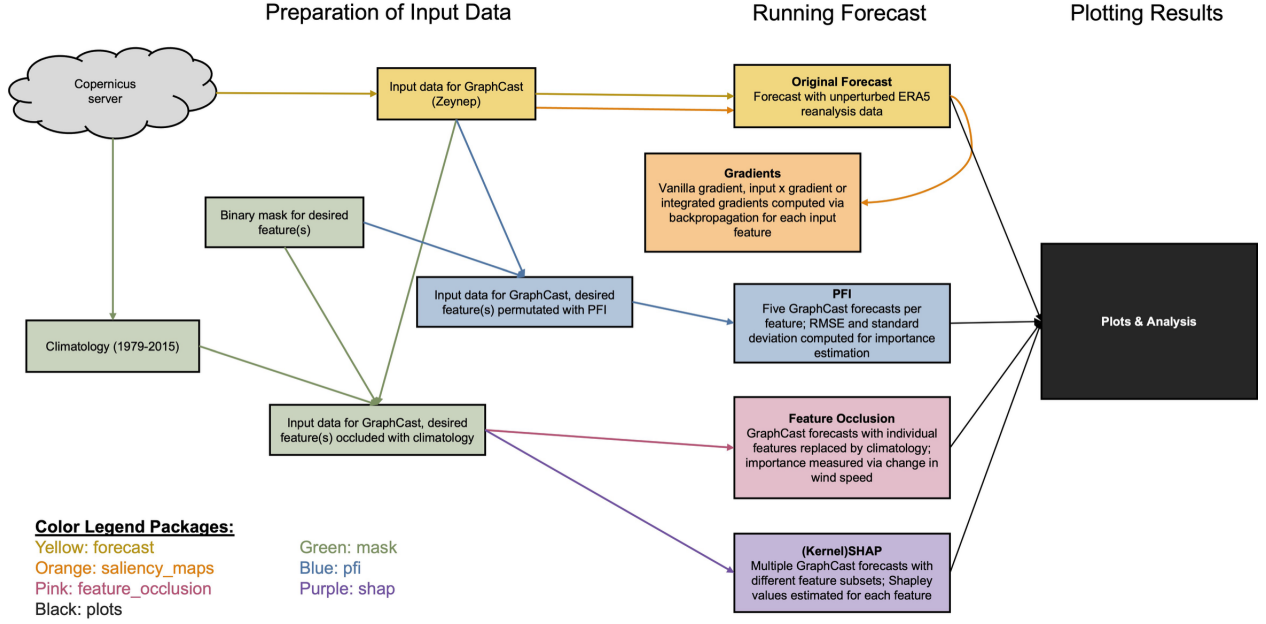


Figure 6: Overview of the forecasting and attribution pipeline.

Input data is retrieved from the Copernicus Climate Data Store and preprocessed for the GraphCast model. Different explainability methods (pfi, feature occlusion, and shap) are applied to modified input variants using either masking or permutation strategies. Forecasts are run with perturbed data and compared to the original forecast to estimate the contribution of individual features. Additionally, gradient-based methods compute the sensitivity of the forecast output with respect to input features via backpropagation, without modifying the input data. The results are visualized using dedicated plotting routines. Color coding corresponds to the respective implementation packages.

4.3.1. Forecast Package

The forecast package (Figure 6, yellow) is responsible for preparing meteorological input data and executing GraphCast model forecasts. It comprises two submodules: `input_preparation`, which retrieves and processes ERA5 reanalysis data into a model-compatible format, and `run_forecast`, which loads a pre-trained GraphCast checkpoint and performs autoregressive inference.

In `input_preparation`, ERA5 pressure-level and surface data are retrieved from the Copernicus Climate Data Store via the CDS API. Downloads are dynamically structured by date and forecast step configuration, allowing forecasts that span multiple days and temporal resolutions. The module handles the extraction and merging of pressure-level and surface data, computes auxiliary variables such as surface geopotential, enriches the dataset with a land-sea mask, and formats the inputs to match GraphCast’s specific requirements.

The `run_forecast` module handles the full inference pipeline using official GraphCast model checkpoints. This includes loading normalization statistics from the public DeepMind GCS bucket, reconstructing the prediction pipeline using GraphCast's autoregressive model wrapper, and executing forecasts using JAX with Haiku-based model definitions.

4.3.2. Mask Package

The mask package (Figure 6, green) implements the logic for generating and applying feature-level perturbation masks based on climatological baselines. It enables targeted attribution studies by isolating the effect of selected atmospheric regions, pressure levels, variables, or time steps on the forecast outcome.

The process begins with `download_mask_data`, which retrieves ERA5 reanalysis data for a specific calendar day across multiple years. Both surface-level and pressure-level variables are downloaded for fixed UTC hours and unzipped into NetCDF format. This data forms the basis for constructing a representative climatology.

The climatology module aggregates these multi-year snapshots into a time-resolved daily climatology. For each input hour, the module computes the average field values across years, harmonizes naming conventions to match GraphCast's input format, and performs longitude normalization. The resulting climatology serves as a physically meaningful replacement for masked inputs.

The `generate_mask` module creates binary masks indicating which features are to be replaced by the climatology baseline. It supports fine-grained specification of mask conditions, including variable selection, vertical levels, spatial regions (e.g., storm cores, jet stream locations), and specific time steps. Regions may be static (predefined bounding boxes) or dynamic, such as upper-level jet stream coordinates computed per time step based on wind speed thresholds. The generated mask aligns structurally with the forecast input dataset.

Finally, `apply_mask` replaces the masked regions in the input dataset with corresponding values from the climatology. The replacement operates on a variable-by-variable basis, ensuring shape consistency and propagating coordinate metadata. A helper function also validates that the

climatology contains all necessary variables, inserting placeholders when needed to maintain compatibility.

4.3.3. Feature Occlusion Package

The `feature_occlusion` package (Figure 6, pink) enables attribution experiments by systematically masking selected input features and replacing them with climatology before running a forecast. This approach helps assess the importance of specific variables, regions, pressure levels, or time steps in the model's predictions.

The main function, `run_forecast_with_mask`, generates a binary mask based on user-defined criteria and applies it to the input dataset. The masked input is then used to run a GraphCast forecast, allowing analysis of how the occlusion affects results.

For larger experiments, `run_multiple_forecasts_with_mask` automates this process across combinations of features, producing reproducible forecasts for each configuration.

4.3.4. Permutation Feature Importance Package

The `pfi` package (Figure 6, blue) implements permutation-based attribution for GraphCast forecasts. It quantifies feature importance by randomly shuffling selected inputs such as variables, pressure levels, or regions within masked areas and observing the impact on prediction accuracy.

The main routine, `run_pfi`, applies permutations across all desired feature combinations and runs multiple randomized forecasts to estimate the stability of results. Mask creation is handled via the existing mask package, and all permutations preserve spatial-temporal coherence except in the perturbed dimensions.

Forecast outputs are saved, and the `evaluate_pfi_folder` utility computes RMSE scores against a ground truth to derive summary statistics per feature, including mean RMSE, standard deviation, and delta from a baseline forecast.

4.3.5. SHAP Package

The `shap` package (Figure 6, purple) implements KernelSHAP explainability for GraphCast forecasts by estimating the marginal contribution of each input feature to the model's final prediction. It focuses on explaining the mean 10 m wind speed over a fixed region (e.g., Northern Germany) at the final timestep of the forecast.

The `run_kernel_shap` function treats each feature (or feature group) as a binary switch, generating masked forecast inputs where features are selectively replaced by climatology. Each perturbed input is passed to the forecast model, and the corresponding regional mean wind speed is recorded. These outputs are fed into a `shap.KernelExplainer`, which computes SHAP values for the given feature set.

4.3.6. Saliency Maps Package

The `saliency_maps` package (Figure 6, orange) implements gradient-based interpretability methods for the GraphCast model, focusing on the attribution of the 10 m wind speed forecast to individual input features at specific geographic locations. It supports three commonly used saliency techniques: vanilla gradients, gradient $\times$ input, and integrated gradients, each computed with respect to the model's scalar output at a predefined forecast lead time and spatial location.

The `compute_gradients` module uses JAX's automatic differentiation to compute these gradients efficiently. It wraps the GraphCast inference pipeline with gradient tracking, enabling backpropagation through the full forecast model. Input variables are converted into JAX-compatible PyTrees and aligned with climatological baselines when needed.

4.3.7. GCS Utilities Package

The `gcs_utils` package (utility shown in Figure 5) provides a lightweight interface for interacting with GCS, enabling seamless upload, download, and listing of files within the GraphCast infrastructure. The client module abstracts authentication and bucket access using environment variables defined in a `.env` file, ensuring compatibility with infrastructure-as-code deployment and reproducible cloud execution.

It exposes a minimal API consisting of `upload_file`, `download_file`, and `list_files`, which are used across the codebase to persist and retrieve model outputs, forecasts, and intermediate artifacts.

4.3.8. Plots Package

The plots package (Figure 6, black) provides visualization tools to evaluate forecast accuracy, interpret attribution results, and support qualitative analysis. It includes routines for plotting scalar fields, derived variables such as wind speed, and various attribution scores. Forecast outputs can be compared to ERA5 reference data, and feature importance can be visualized through bar plots, heatmaps, or saliency maps. The plots are rendered on geographic projections when appropriate, facilitating spatial interpretation across regions and vertical levels. By standardizing visual output across different stages of the pipeline, the package supports efficient analysis and consistent presentation of results.

4.4. Testing

To ensure correctness, stability, and reproducibility across the entire pipeline, a comprehensive suite of unit and integration tests was implemented. Each major module in the Python package is accompanied by corresponding tests.

Unit tests validate individual functions in isolation, using mocks to decouple from external services such as the Copernicus CDS API and Google Cloud Storage. Integration tests verify the correct interaction between components by running higher-level routines, such as climatology computation, feature perturbation, and SHAP value estimation, often using synthetic or real NetCDF datasets.

Special attention was given to edge cases and failure modes, including missing input files, inaccessible model checkpoints, and improperly configured environments. Forecast and attribution outputs are validated for structural and numerical consistency, and tests are executed in a controlled local environment.

4.5. Privacy and Ethics

This project does not process any personal, sensitive, or proprietary data. All datasets used are publicly available from the Copernicus Climate Data Store or published model repositories. The analysis is strictly physical in nature and does not involve human subjects, identifiable individuals, or personal information.

The codebase is developed with transparency and reproducibility in mind, aligning with principles of open science. Where third-party models or datasets are used (such as GraphCast or ERA5), appropriate licensing terms and attribution guidelines are followed. The GraphCast model weights and code are used under the Creative Commons Attribution-NonCommercial-ShareAlike 4.0 and Apache 2.0 licenses, respectively.

No automated decision-making affecting individuals is conducted, and no ethical risks were identified in the development or application of this system. Cloud resources are used responsibly, and all infrastructure provisioning is reproducible and documented.

5. Meteorological Expectations

Based on established meteorological theory and the known characteristics of storm Zeynep, several variables and pressure levels can be identified as particularly relevant for determining surface wind speeds over northern Germany.

Wind speed is the most directly relevant variable in this context. It influences storm development at multiple heights in the atmosphere. At the high-altitude level of 300 hPa, a strong west-to-east jet stream over the Atlantic helped drive the rapid movement of storm Zeynep toward Europe. A more concentrated area of faster winds within this jet stream, known as a jet streak, was located over northern Europe during the event. This jet streak contributed to rising motion in the atmosphere, which in turn helped deepen the storm system below. Closer to the surface, at around 850 hPa, wind speed is linked to the so-called low-level jet stream. This is a band of fast-moving air that often forms on the southern and eastern sides of strong low-pressure systems. It is important because it can bring strong winds closer to the ground and is often associated with local wind maxima near the surface [29], [30].

Geopotential height is another critical variable, with its importance varying by pressure level. At 500 hPa, geopotential height reflects the structure and depth of the mid-tropospheric trough, which was strongly pronounced over northern Europe during the event. At 850 hPa, geopotential correlates to the location and intensity of the surface cyclone and the surrounding pressure gradient zone, both of which are tightly linked to the near-surface wind field [29]. At the surface, the mean sea level pressure reveals a deep low-pressure system over northern Europe and a high-pressure area farther south creating a sharp pressure contrast. This contrast enhanced the geostrophic winds (those driven by pressure and Earth's rotation) across northern Germany, contributing to the damaging surface wind speeds observed [29], [31]. Notably, the high-altitude jet streak was spatially aligned with the surface pressure gradient. This vertical alignment is typical of so-called baroclinic systems, where differences in temperature and pressure at various levels support strong storm development and jet formation. In line with these dynamics, it is reasonable to expect that geopotential influences on surface wind forecasts would extend from 500 hPa down to the surface, especially in regions near the Storm Core and pressure gradient zones.

Finally, temperature and humidity also influence storm dynamics. Temperature gradients fuel baroclinicity, which helps organize storm structure and frontal systems, while humidity supports the release of latent heat, adding energy to the system. Although these variables are not the primary focus in this study, they may play a supporting role in localized surface wind variability [31], [32].

In summary, GraphCast is expected to attribute surface wind forecasts primarily to wind speed and geopotential height patterns. Areas of special interest include the Atlantic Jet Stream, the Western-Europe Jet Streak, the Storm Core, the low-pressure zone over northern Europe, and the pressure gradient zone between northern and southern Europe. These expectations are listed in Table 1 and illustrated in Figure 7 for the inputs used to obtain the forecast at 22:00 UTC, 18 February 2022.

Table 1: Meteorological variables, levels and regions expected to influence surface wind forecasts in northern Germany during storm Zeynep.

Variable	Relevant Region	Typical Pressure Level Range
Wind speed	Atlantic Jet Stream	~ 300 hPa
	Western-Europe Jet Streak (upper-level)	~ 300 hPa
	Western-Europe Jet Streak (low-level)	~ 850 hPa – surface
Geopotential	Low Pressure Area over Northern Europe	~ 500 hPa
	Storm Core	~ 500 hPa – surface
	Pressure Gradient (interrelated with the Western-Europe Jet Streak)	~ 500 hPa – surface

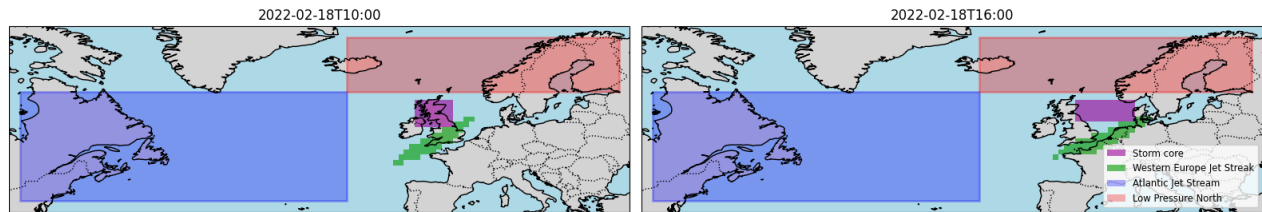


Figure 7: Spatial areas of the GraphCast inputs expected to contribute to the high winds in northern Germany observed for Zeynep

Colors indicate: Atlantic Jet Stream (blue), Western Europe Jet Streak/ low level jet stream interrelated with the pressure gradient (green), Storm Core (purple), and Low-Pressure North (red). Left panel: 18 February 2022 10:00 UTC; right panel: 18 February 2022 16:00 UTC. These maps correspond to inputs for the GraphCast forecast at 22:00 UTC on the 18 February 2022.

6. Results

6.1. Analysis of Forecasts

To assess the performance of GraphCast during Storm Zeynep on 18 February 2018, predicted 10 m wind speeds were compared against ERA5 reanalysis data. The evaluation focused on both spatial and temporal accuracy, using the ERA5 dataset as the observational reference.

At the target time of 22:00 UTC, spatial comparisons were conducted for the GraphCast_operational and GraphCast_small variants. As shown in Figure 8, both models accurately reproduce the overall wind field, including the high wind zone along the North Sea coast. However, a consistent underestimation of wind speeds is observed in the northern coastal region in both variants.

For GraphCast_small (1.0° resolution), the mean predicted wind speed over northern Germany (53°N-55°N, 8°E-10°E) was 13.36 m/s, compared to 14.72 m/s in the ERA5 data. This corresponds to a negative bias of -1.36 m/s, based on 9 grid points. The MAE was 1.47 m/s, while the RMSE was 2.03 m/s.

The GraphCast_operational model (0.25° resolution) demonstrated a predicted mean wind speed of 14.24 m/s, compared to 15.22 m/s in the ERA5 reference, yielding a bias of -0.98 m/s. This evaluation was based on 81 grid points. The resulting MAE was 1.07 m/s, and the RMSE was 1.59 m/s, indicating better overall agreement with observations.

In addition to the spatial analysis, temporal evolution of wind speed was also evaluated throughout the day. Figure 9 presents the wind speed progression averaged over the same northern Germany region, comparing ERA5 with GraphCast_small in both single-step and multi-step prediction modes. The single-step mode uses two prior observations for each 6-hour forecast, whereas the multi-step mode recursively feeds its own outputs back into the model to extend the forecast horizon, starting from 11:00 and 17:00 UTC on 17 February. Both methods capture the overall trend and the timing of peak wind speeds, although the forecast tends to slightly underestimate peak values during storm onset.

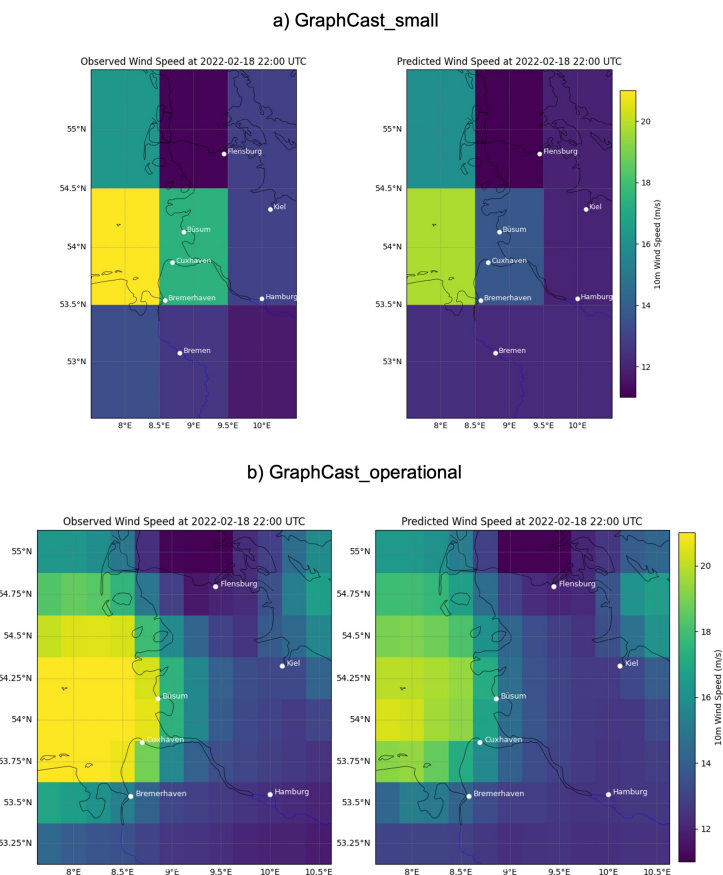


Figure 8: Wind speed predictions of different GraphCast models vs. ERA5 data

a) GraphCast_small and b) GraphCast_operational wind speed predictions vs. ERA5 observations at 22:00 UTC on 18 February 2022. Spatial resolution and intensity of wind fields are shown over northern Germany.

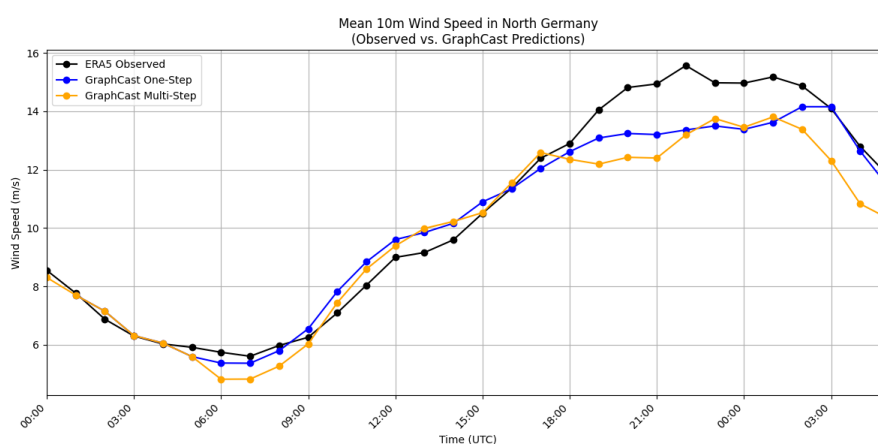


Figure 9: Temporal evolution of forecasted 10 m wind speed during Storm Zeynep

Mean 10 m wind speed over northern Germany (53°N–55°N, 8°E–10°E) on 18 February 2022, comparing ERA5 reanalysis data (black) with GraphCast_small forecasts in single-step (blue) and multi-step (orange) configurations. Multi-step forecasts were initialized at 11:00 to 17:00 UTC on 17 February. The plot highlights model performance in capturing the wind speed evolution during storm Zeynep.

6.2. Explainability of the Forecast

To interpret how GraphCast predicts surface wind speeds during Storm Zeynep, a structured explainability framework was applied that combines complementary attribution methods. The analysis begins with global, model-agnostic techniques – feature occlusion, pfi, and KernelSHAP – which identify the most influential meteorological variables for the forecasted 10 m wind speed (Section 6.2.1). Building on these results, a second step applies region- and pressure-level feature occlusion to examine how key atmospheric drivers, such as geopotential height and wind speed, contribute across specific meteorological structures and vertical layers (Section 6.2.2). Finally, integrated gradients are used to generate fine-grained, spatially explicit attribution maps, highlighting localized patterns of influence in the input fields (Section 6.2.3).

6.2.1. Variable Importance via Model-Agnostic Methods

To investigate which input variables most significantly influenced the prediction of 10 m wind speed over northern Germany during Storm Zeynep, three model-agnostic explainability methods were applied: feature occlusion, pfi, and KernelSHAP. Each method quantifies the contribution of individual input variables to the model’s forecast, using the evaluation region 53°N-55°N and 8°E-10°E as the target. The attribution methods were applied to a single-step GraphCast forecast at 22:00 UTC, using input states at 10:00 and 16:00 UTC.

The analysis included both surface and atmospheric predictors. Surface variables comprised 10 m wind speed (decomposed into u- and v-components), mean sea level pressure, 2 m temperature, and 6 h accumulated total precipitation. Atmospheric variables included wind speed, geopotential, temperature, specific humidity, and vertical velocity across all available pressure levels.

Feature occlusion evaluates the sensitivity of the forecast to individual variables by replacing selected inputs with climatological values, while pfi quantifies importance by measuring the increase in prediction error resulting from random permutation of variable values. In contrast, KernelSHAP approximates the Shapley values for each variable, capturing its marginal contribution to the predicted output relative to other features.

Figure 10 presents the resulting feature rankings. Panel (a) shows the impact of occluding each variable on forecast wind speed. Panel (b) displays the increase in MAE due to random permutation of each variable. Panel (c) shows the SHAP values, where positive contributions are colored in blue and negative contributions in red.

All three methods consistently identify wind speed, 10 m wind speed, geopotential, and mean sea level pressure as the most influential features contributing to high surface wind speeds in the region of interest. Notably, vertical velocity was found to be the least important variable across all methods. In fact, masking or permuting vertical velocity occasionally led to slight increases in predicted wind speed, suggesting a negligible or even mildly counteractive role for this variable in this specific context. The attribution of temperature, specific humidity, 2 m temperature, and precipitation varies between the methods, though none were among the dominant drivers.

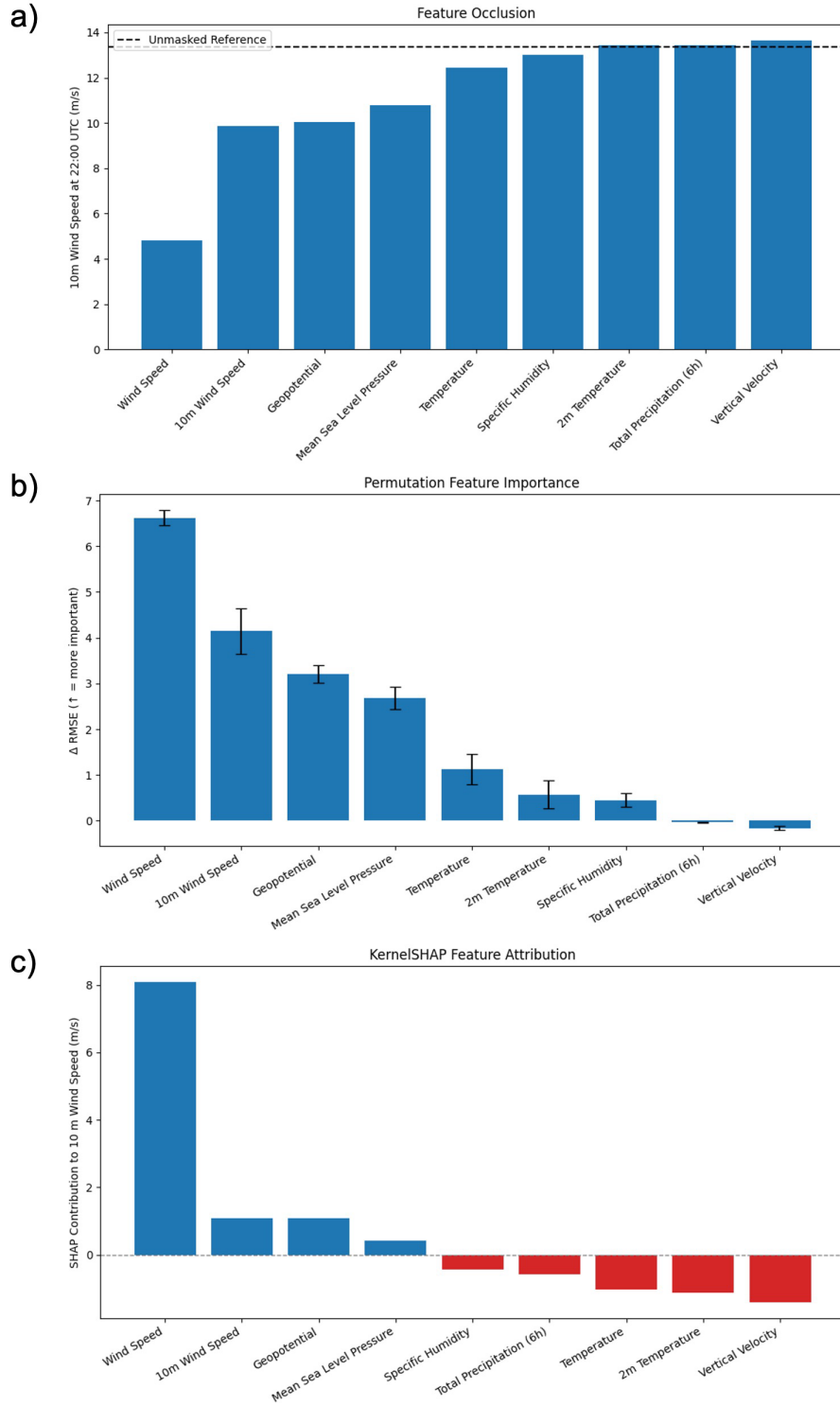


Figure 10: Global feature importance of weather variables analyzed with model-agnostic perturbation methods

Global feature importance for surface wind prediction over northern Germany (53°N-55°N, 8°E-10°E), based on a single-step GraphCast forecast with input times at 10:00 and 16:00 UTC and a target time of 22:00 UTC on 18 February 2022. Importance scores were computed using (a) feature occlusion, (b) pfi, and (c) KernelSHAP. All three methods consistently highlight wind speed, 10 m wind speed, geopotential, and mean sea level pressure as key drivers.

6.2.2. Feature Occlusion Analysis Across Regions and Pressure Levels

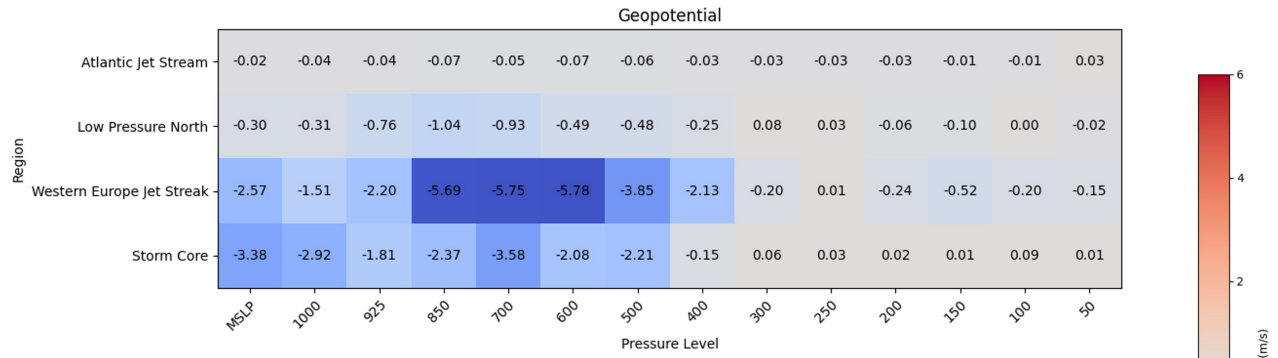
To evaluate the spatial and vertical relevance of key atmospheric predictors, a region- and pressure-specific feature occlusion analysis was conducted for the variable's geopotential height and wind speed. Surface variables, specifically 10 m wind speed and mean sea level pressure, are also included to assess near-surface sensitivity. The analysis focusses on four meteorologically significant regions: the Atlantic Jet Stream, the Low-Pressure Area over Northern Europe, the Western Europe Jet Streak, and the Storm Core. For each region, input data was selectively occluded at individual pressure levels for both input times (10:00 and 16:00 UTC). The resulting change in forecasted 10 m wind speed over northern Germany (53°N-55°N, 8°E-10°E) at 22:00 UTC was then recorded.

Figure 11 displays the resulting attribution heatmaps. Panel (a) shows the impact of occluding geopotential height, and panel (b) illustrates the same for wind speed. Color intensity indicates the magnitude and direction of forecast change: darker blue values represent a reduction in predicted surface wind speed, implying higher importance of the occluded variable. In contrast, red values indicate an increase in forecasted wind speed, suggesting the variable may have introduced noise or had a dampening effect on forecast accuracy.

For geopotential height, the strongest influence appears in the Western Europe Jet Streak region, with notable occlusion effects from 400 hPa to the surface, and a peak contribution centered near 700 hPa. The Storm Core also exhibits consistent sensitivity, particularly between 500 hPa and the surface. The Low-Pressure North region shows moderate importance, especially between 500 and 925 hPa. Nearly no influence is observed in the Atlantic Jet Stream region.

Wind speed attribution is generally weaker in magnitude. The most pronounced sensitivity again occurs in the Western Europe Jet Streak, with peaks near the surface and between 400-500 hPa. The Storm Core shows moderate impact at 700-850 hPa, while the Atlantic Jet Stream and Low-Pressure North regions contribute minimally.

a)



b)

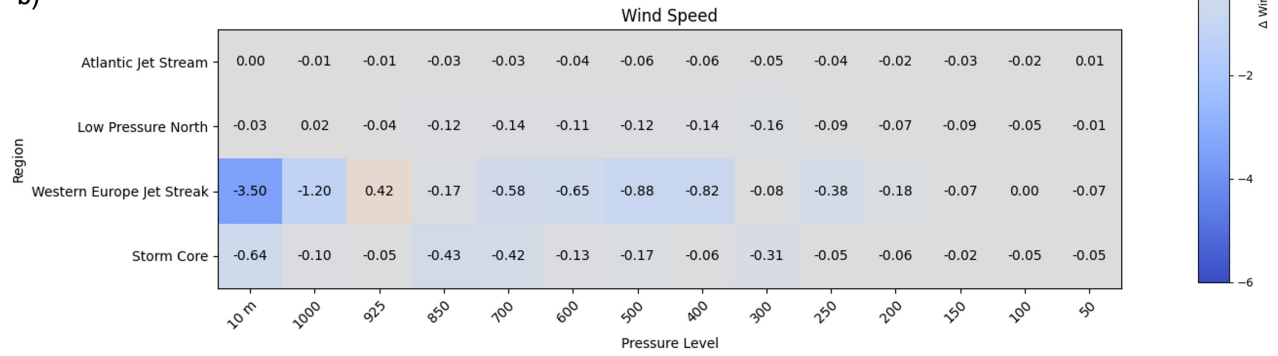


Figure 11: Feature occlusion analysis across pressure levels and meteorological regions

Change in predicted 10 m wind speed over northern Germany (53°N - 55°N , 8°E - 10°E) at 22:00 UTC, resulting from region- and pressure-specific occlusion of input variables at 10:00 and 16:00 UTC on 18 February 2022.

Panel (a) shows the change in wind speed in northern Germany (53°N - 55°N , 8°E - 10°E) when occluding geopotential height, and panel (b) shows the same for wind speed. Each cell represents a pressure-level occlusion within one of four regions of interest: Atlantic Jet Stream, Low Pressure North (northern Europe), Jet Streak over Western Europe, and the Storm Core. Colors indicate the change in wind speed compared to the unperturbed forecast, with blue indicating decreased and red indicating increased wind speed.

6.2.3. Gradients for Spatial Analysis

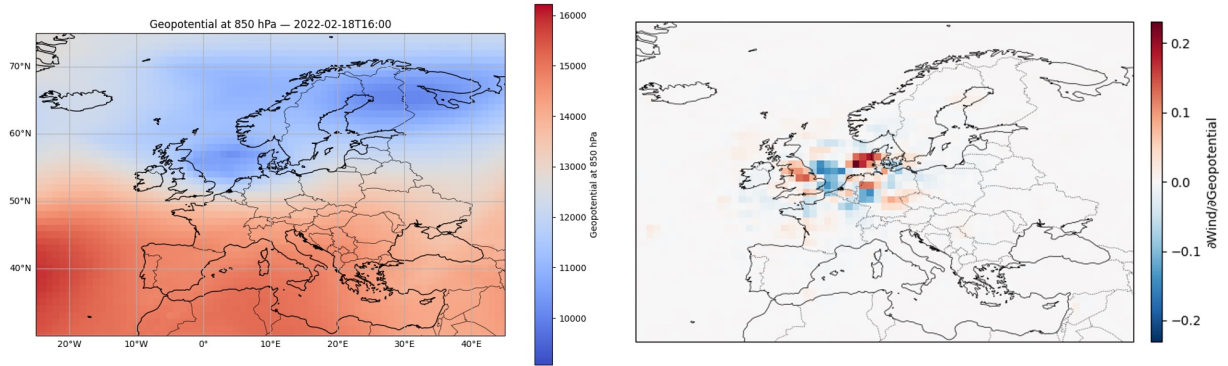
To determine which spatial regions and atmospheric structures most strongly influenced the forecasted 10 m wind speed at a specific location, integrated gradients attribution was applied to GraphCast input fields. The analysis focused on the forecast grid point at 54.0°N, 9.0°E in northern Germany, a region where surface wind impacts from storm Zeynep were particularly strong. Model inputs from both 10:00 UTC and 16:00 UTC on 18 February 2022 were used to predict wind speed at 22:00 UTC.

Figure 12 displays attribution maps for 16:00 UTC and two key atmospheric variables: geopotential height at 850 hPa (panel a) and the zonal wind (u-component) at 300 hPa (panel b).

In panel (a), the geopotential field shows a strong meridional pressure gradient across England, mid-Germany to Poland, with lower heights over the North Sea and higher values over southern Europe. The corresponding saliency map highlights the region between eastern England and northern Germany, near the core of the low-pressure system and around the pressure gradient, as the most influential for the 10 m wind forecast.

In panel (b), the 300 hPa zonal wind field reveals the structure of the upper-level jet stream, extending from the Bay of Biscay across England toward northern Germany. The attribution map shows a clear saliency band aligned with this jet axis, particularly over the English Channel to northern Germany.

a) Geopotential Saliency at 850 hPa for 10 m Wind Speed (16:00 UTC)



b) U Component of Wind Saliency at 300 hPa for 10 m Wind Speed (16:00 UTC)

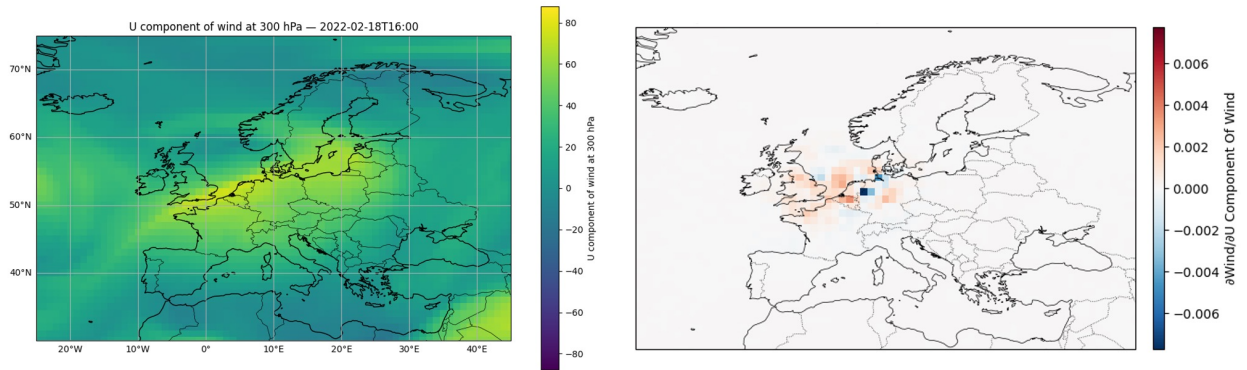


Figure 12: Spatial attribution of forecasted surface wind using Integrated Gradients

Integrated Gradients attribution for 10 m wind speed at 54.0°N, 9.0°E, based on GraphCast input at 16:00 UTC and forecast target time 22:00 UTC on 18 February 2022. Panel (a) shows the raw geopotential height at 850 hPa (left) and the corresponding attribution map (right). Panel (b) presents the zonal wind (u-component) at 300 hPa and its attribution. Red and blue shading in the saliency maps indicates positive and negative contributions relative to the climatological baseline.

7. Discussion

7.1. Analysis of Forecast

The GraphCast forecast of the two models GraphCast_small and GraphCast_operational on the 18th of February 2022 are compared to the ERA5 reanalysis data. Both GraphCast variants accurately capture the overall spatial structure of the 10 m wind field during storm Zeynep, particularly the high wind zone along the North Sea coast. However, the systematic underestimation of peak wind speeds, especially in the northern coastal region, highlights some model limitations.

GraphCast_operational (0.25° resolution) shows better agreement with ERA5 in both bias (-0.98 m/s) and RMSE (1.59 m/s) compared to GraphCast_small (-1.36 m/s bias; 2.03 m/s RMSE). Despite this higher accuracy, GraphCast_small was selected for subsequent experiments due to its significantly lower computational requirements.

The comparison of wind speed in GraphCast_small (Figure 9) displays that the model reproduces the overall temporal trend well in both single-step and multi-step forecast settings. While multi-step forecasts naturally degrade due to error accumulation, the trend and timing of peak wind events are still preserved. The slight underestimation during storm onset suggests that GraphCast may underpredict rapid intensification phases.

7.2. Explainability of the Forecast

While the previous section demonstrates that GraphCast provides a reasonably accurate forecast of surface wind speeds during storm Zeynep, particularly in capturing the overall spatial structure, some systematic underestimation of wind maxima, especially along the North Sea coast, is observed. This motivates a deeper investigation into the model's internal mechanisms to assess whether its predictions are not only accurate, but also physically consistent and interpretable.

To that end, explainability methods are applied to examine which atmospheric variables and spatial structures most strongly influenced the forecast. This interpretability analysis seeks to answer

whether the features driving GraphCast’s predictions align with established meteorological theories.

As outlined in the chapter 5. Meteorological Expectations, the primary dynamical drivers of near-surface wind speed during extratropical storm events such as Zeynep, are wind speed and geopotential height. For wind speed, key regions of interest include the Atlantic Jet Stream near 300 hPa, as well as the Western Europe Jet Streak, which manifests both at upper levels (~300 hPa) and at lower levels (~850 hPa to the surface). In terms of geopotential height, important features include the low-pressure system over northern Europe, typically exerting its influence near 500 hPa, as well as the Storm Core and the associated surface pressure gradient, both of which exhibit vertical coupling from approximately 500 hPa down to the surface.

Interpreting GraphCast, however, poses unique challenges. While the model is based on graph neural networks, its public interface exposes only limited internal information, restricting access to intermediate representations such as node embeddings, edge weights, or attention mechanisms. This limits the applicability of GNN-specific interpretability tools and necessitates alternative strategies. To address this, a layered attribution strategy is adopted that combines multiple explainability methods, each suited to different levels of abstraction. The selected techniques (feature occlusion, pfi, KernelSHAP, and integrated gradients) were chosen not only for their compatibility with GraphCast’s restricted access but also for their complementary analytical strengths. Model-agnostic methods provide robust global insights into variable relevance, while region- and pressure-specific occlusion enables targeted attribution within meteorologically significant domains. Integrated gradients, in turn, offer fine-grained spatial attribution that preserves the coherence of atmospheric structures. Together, these methods form a structured interpretability framework that balances feasibility, interpretive depth, and meteorological relevance.

7.2.1. Variable Importance via Model-Agnostic Methods

This section aimed to assess which meteorological input variables most strongly influenced GraphCast’s 10 m wind speed forecasts during Storm Zeynep using three model-agnostic explainability methods: feature occlusion, pfi, and KernelSHAP. Each method adopts a different

approach to estimate variable relevance: feature occlusion replaces selected inputs with climatological means; pfi randomly shuffles values, thereby preserving statistical properties while disrupting feature associations; and KernelSHAP approximates each feature's marginal contribution by considering coalitions of inputs relative to a baseline forecast.

All methods consistently identified wind speed, 10 m wind speed, geopotential height, and mean sea level pressure as the most impactful predictors. This finding aligns well with meteorological expectations established earlier, where large-scale jet dynamics, mid-to-lower tropospheric geopotential gradients, and near-surface pressure fields were expected to play central roles. In contrast, thermodynamic variables such as temperature, specific humidity, and accumulated precipitation had limited explanatory value.

Despite general agreement among methods, each approach comes with inherent methodological differences that shape their results.

Feature occlusion may introduce discontinuities in the meteorological context when entire fields are replaced by climatology. Such disruptions can produce artifacts, especially in variables with strong spatial correlations, and may exaggerate or suppress importance depending on how well the remaining inputs can compensate.

Pfi avoids such artificial discontinuities by randomly shuffling feature values across samples, preserving their marginal distributions. However, this strategy comes with its own limitations. Pfi is sensitive to feature correlation: when variables are collinear, as often occurs in multilevel meteorological data, shuffling one feature breaks meaningful joint patterns, potentially producing unrealistic or physically implausible input combinations. This can bias the importance estimates, either inflating or deflating them depending on the redundancy among correlated variables. Moreover, when multiple correlated features share predictive power, pfi tends to distribute importance across them, making it harder to identify truly dominant drivers. Pfi also quantifies importance in terms of prediction error but does not capture the direction of a feature's influence. Moreover, like feature occlusion, nonlinear interactions between features are not captured, as each variable is tested in isolation. [19].

KernelSHAP, in contrast, explicitly estimates each feature's marginal contribution by simulating coalitions of features. It is better suited to capturing interactions and synergies between variables.

However, it also has notable limitations. First, KernelSHAP assumes feature independence and samples from marginal distributions, which can result in unrealistic data points when features are correlated, potentially distorting the estimated contributions. Second, it is computationally expensive and scales poorly to high-dimensional inputs, such as multilevel meteorological fields [19]. In this study, computational constraints limited the number of KernelSHAP samples to 100, which likely affected the convergence and stability of the attributions.

A further issue emerged in the form of attribution completeness. Ideally, the sum of SHAP values across features should match the difference between the model output and the baseline prediction. However, in this case, the cumulative SHAP value (6.1 m/s) explained only part of the total forecast deviation (13.2 m/s; $14.7 \text{ m/s (forecast value)} - 1.5 \text{ m/s (baseline value)}$). This shortfall suggests that KernelSHAP may struggle to faithfully reconstruct the full predictive process of highly nonlinear models like GraphCast, particularly when relying on a small number of coalitions.

To mitigate these limitations, conditional SHAP variants have been proposed that replace marginal sampling with conditioning on the observed feature subset. Aas et al. (2021) show that such methods yield more accurate and stable attributions when input variables are dependent [33]. A related approach is the Conditional Expectation Network, which trains a neural surrogate to directly learn conditional expectations for SHAP value estimation [34]. While these methods were not used in this study, they represent promising avenues for improving attribution quality in future applications of SHAP to high-dimensional weather models.

In sum, all three techniques provide coherent and meteorologically plausible insights into variable importance, confirming that GraphCast's surface wind forecasts in northern Germany for Zeynep rely on wind speed and geopotential.

7.2.2. Feature Occlusion Analysis Across Regions and Pressure Levels

To further investigate the spatial and vertical importance of atmospheric drivers, the two key variables, geopotential height and wind speed, were subjected to feature occlusion analysis across defined meteorological regions and pressure levels (Figure 11).

For geopotential height, the Western Europe Jet Streak emerges as the most influential region. This result aligns closely with meteorological expectations, as this region spans the area of the strongest pressure gradient during Storm Zeynep, which is known to be a critical driver of surface wind acceleration. The vertical attribution extends from approximately 400 hPa to the surface, consistent with the anticipated range (~500 hPa to the surface). Similarly, the Storm Core, encompassing the cyclone center and surrounding gradient zone, exhibits substantial attribution, especially between 500 hPa and the surface. This finding also conforms well to the theoretical expectation, given the central role of the surface cyclone in shaping near-surface winds. The Low-Pressure North shows moderate importance, peaking around 850 hPa and extending between 500 and 925 hPa. This is slightly lower than the expected influence around 500 hPa.

Wind speed occlusion reveals the Western Europe Jet Streak as the main contributor, with peaks in sensitivity both near the surface and at 400-500 hPa. These levels correspond well to the low-level jet stream and upper-level jet streak, respectively key dynamic features associated with surface wind maxima. The Storm Core shows moderate attribution centered around 700-850 hPa, suggesting that wind speed in this region may primarily act through low- to mid-level jet dynamics. The Low-Pressure North region exhibits minimal contribution from wind speed, which is consistent with its expected role as a source of geopotential forcing rather than a region of dynamically significant wind fields.

The Atlantic Jet Stream shows negligible influence for both geopotential and wind speed. This is likely attributable to the temporal setup of the experiment, in which the input data was drawn from 12 hours prior to the target forecast time. The Atlantic Jet Stream's contribution may have occurred earlier in the forecast sequence, primarily influencing storm evolution upstream before entering the domain of interest.

Interestingly, wind speed generally shows weaker attribution magnitudes compared to geopotential height, despite being among the globally most important variables. The impact of wind speed may be distributed across multiple pressure levels and spatial regions, making its attribution appear diluted when examined in isolation. To explore this further, the individual occlusion impact of pressure levels and regions for wind speed were analyzed (Appendix Figures A2 and A3). Both plots show enhanced values for the occluded pressure levels or regions. Furthermore, they confirm that the greatest sensitivity occurs within the 300-850 hPa range, with a peak at 400 hPa and at the

surface. Regionally, the Western Europe Jet Streak stands out with the strongest attribution scores. Taken together, these findings might suggest that wind speed attribution is characterized by synergetic effects across a broader vertical and horizontal domain, in contrast to the more spatially localized and pressure-focused influence of geopotential.

In conclusion, the feature occlusion analysis validates the importance of both the Western Europe Jet Streak and the Storm Core in determining surface wind speeds over northern Germany. The influence of these features was evident for both geopotential height and wind speed, and at pressure levels consistent with meteorological theory. Conversely, the northern low-pressure zone and Atlantic Jet Stream played comparatively minor roles, with the latter likely influencing earlier stages of storm development beyond the temporal window analyzed in this experiment. Comparatively lower attribution values are observed for wind speed that may reflect on its more synergetic role in the forecast process.

7.2.3. Gradients for Spatial Analysis

The integrated gradients analysis offers spatially explicit insight into how GraphCast leverages atmospheric structures to predict surface wind speed at a specific location.

The attribution map for geopotential height at 850 hPa reveals strong positive contributions originating from the region between eastern England and northern Germany. This corresponds closely to the zone of enhanced pressure gradients and the Storm Core. These findings are consistent with theoretical expectations that the pressure gradient near the surface and the low-pressure area in the Storm Core are drivers of the near-surface wind acceleration in Zeynep.

The zonal wind field at 300 hPa and its associated attribution highlight the structure of the upper-level jet stream. The saliency band closely follows the jet axis from the English Channel across toward northern Germany, suggesting that the model associates upper-level dynamical forcing with enhanced surface winds at the forecast location. This alignment is meteorologically plausible, as the jet streak promotes upper-level divergence and vertical motion, which can deepen the surface low, intensify pressure gradients, and thereby strengthen surface wind speeds.

An important strength of the integrated gradients approach lies in its ability to capture these spatially coherent patterns of influence. Unlike feature-wise occlusion or permutation methods, which quantify variable importance in isolation, gradient-based attribution considers how small changes in each input contribute to the prediction, relative to a climatological baseline. Compared to raw gradients or the gradient $\times$ input approach, integrated gradients provide a more stable and theoretically grounded attribution method. By integrating gradients along a continuous path from a baseline to the actual input, they mitigate issues such as gradient saturation and fragmented saliency maps, which frequently arise in highly nonlinear architectures [24].

However, as with many local attribution techniques, integrated gradients are not without limitations. The method assumes that the model behaves smoothly along the interpolation path from baseline to input, which may not always hold in practice. Studies have shown that gradient-based attributions, including integrated gradients, can be sensitive to input shifts and adversarial perturbations, sometimes leading to unstable or misleading explanations despite unchanged model outputs [35], [36]. Additionally, the numerical attribution values themselves should not be overinterpreted in isolation. Their utility lies primarily in their spatial distribution and relative patterns, rather than absolute magnitude.

Nevertheless, in this study, the spatial attribution patterns from integrated gradients align well with meteorological expectations, such as the enhanced pressure gradient and the upper-level jet streak, supporting the view that GraphCast’s forecasts are informed by physically meaningful features and that its internal representations reflect plausible atmospheric dynamics.

Furthermore, the integrated gradients results show strong agreement with the feature occlusion analysis: both methods independently highlight the Western Europe Jet Streak and the Storm Core as key regions influencing surface wind speeds, while attributing only minor or no relevance to the Atlantic Jet Stream within the forecast window. This convergence of findings suggests that the manually defined meteorological regions used in the occlusion experiments capture the same dominant structures that emerge from data-driven gradient attributions. The consistency across methods increases confidence that the identified drivers are both robust and physically meaningful, providing a cross-validation between model-agnostic perturbation techniques, gradient-based explanations, and domain knowledge.

7.2.4. Explaining GraphCast: Challenges and a Structured Interpretation Strategy

While the previous sections demonstrate that GraphCast’s forecasts can be meaningfully interpreted using a combination of attribution techniques, several limitations arise when attempting to apply standard explainability tools. These challenges are closely tied to the complexity of the GraphCast architecture and the high dimensionality of meteorological data, both of which constrain the interpretability methods that can be feasibly used in this context.

Although GraphCast is based on GNNs, it cannot be easily analyzed with common GNN-specific interpretability methods such as GNNExplainer, GraphSHAP, or SubgraphX. One reason is that GraphCast is not a single, standalone GNN model. Instead, it is composed of multiple interconnected GNN blocks that process spatiotemporal weather data in a recursive fashion. These blocks operate over a large input graph, representing global atmospheric states across pressure levels and physical variables. This architectural complexity makes it difficult to isolate the kind of subgraphs or localized attention paths that many GNN explainer methods rely on.

In addition, most GNN-specific explanation methods rely on access to intermediate model internals such as edge weights, node embeddings, or attention maps. These components are not readily exposed in the public version of GraphCast. While it is possible to access inputs, outputs, and even gradients by modifying the model pipeline using JAX and Haiku, more detailed introspection – such as tracing information flow across nodes or analyzing internal graph representations – requires forking and altering the underlying source code. As such, the model effectively functions as a black box for most practical purposes, and this restricts the use of many white-box interpretability tools. This limitation motivates the use of model-agnostic and gradient-based methods as the primary means of attribution analysis in this study.

A further challenge is the sheer scale of the input feature space. GraphCast processes thousands of grid points across multiple pressure levels and atmospheric variables. This high dimensionality makes methods like LIME or SHAP unstable unless input features are coarsened into semantically meaningful units. However, such aggregation is not trivial and requires domain knowledge to group features into larger, meteorologically interpretable structures. For example, rather than explaining individual grid points, this study defines larger feature regions such as the Western Europe Jet Streak or the Storm Core and analyzes their contribution at selected pressure levels.

Given these constraints, this thesis adopts a structured, layered approach to interpretability. The central idea is to open the black box not through a single explainer, but by combining complementary methods that operate at different levels of abstraction. The first layer consists of global model-agnostic techniques (feature occlusion, pfi, and KernelSHAP) that identify which input variables are most relevant overall. The second layer applies targeted occlusion to meteorologically significant regions and pressure levels, enabling finer-grained spatial and vertical attribution. The third layer uses integrated gradients to produce localized spatial attribution maps, highlighting coherent atmospheric patterns that influence specific forecast locations.

This three-stage interpretability strategy balances methodological feasibility with physical insight. It acknowledges the practical limitations of current tools while offering a generalizable approach to interpreting complex weather models. Importantly, the strategy is designed to be extendable: although this study focuses on surface wind forecasts during a specific storm event, the same framework could be applied to other variables, regions, or models.

7.3 Outlook

This study demonstrates that GraphCast, even in its reduced-resolution variant (GraphCast_small), can generate physically plausible surface wind forecasts and reveal meaningful attribution patterns through explainability techniques. However, several limitations and opportunities for future work remain.

7.3.1. Model Fidelity and Resolution

While the smaller GraphCast_small model was chosen for computational feasibility, the operational GraphCast and most probably the normal GraphCast version exhibit higher accuracy. Future work could benefit from applying the same explainability framework to the full-resolution model, thereby enhancing realism and reducing uncertainty in attribution outcomes.

An open research question is how model resolution affects the interpretability of XAI methods. For example, do feature attribution patterns (e.g., variable importance or saliency maps) become

more consistent and physically meaningful when derived from a higher-resolution GraphCast variant? Investigating this relationship could help determine whether XAI conclusions drawn from GraphCast_small are fully transferable to operational models or if resolution-dependent biases exist.

7.3.2. Forecast Horizon

This study focused on a single-step forecast ($t - 6 \text{ h}$, $t \rightarrow t + 6 \text{ h}$), using inputs only up to 12 hours before the target time. While this setup isolates the immediate drivers of the forecast, it may overlook earlier influences, such as the Atlantic Jet Stream, which typically exerts its effect earlier in the storm development process. Applying the explainability framework to multi-step recursive forecasts could help identify temporally distant but causally relevant structures. Such analyses would enrich the understanding of how GraphCast propagates meteorological signals across time.

7.3.3. Improving Region-Based Occlusion Strategies

Feature occlusion over spatial regions and pressure levels is informative but constrained by the need to predefine regions. In this work, regional masks were guided by meteorological reasoning, but ultimately required manual selection, introducing a degree of subjectivity. A more scalable and objective strategy could involve denser occlusion sampling over the full domain, either grid-pointwise or using automated clustering of meteorological structures (e.g., jet axes). Although such methods would significantly increase computational cost, they would remove dependence on hand-crafted regional masks and offer more comprehensive coverage.

Another promising direction would be to automate the comparison between gradient-based and occlusion-based attributions. For example, regions highlighted by integrated gradients could be used as masks for targeted FO experiments, allowing a more systematic cross-validation of the two approaches. This could reveal whether the predefined meteorological regions align with data-driven saliency patterns or if important structures are being overlooked. Such a hybrid approach would enhance confidence in attribution results and reduce the reliance on manually defined regions.

7.3.4. Conditional SHAP and Neural Surrogates

One limitation of the KernelSHAP method used in this study is its assumption of feature independence, which can lead to unrealistic coalitions and distorted attributions when features are correlated. This is particularly problematic in meteorological applications, where variables such as geopotential height, wind speed, and temperature are often strongly interdependent across space and time.

Future research could explore conditional SHAP methods, which sample from the conditional rather than marginal distribution of masked features.

Aas et al. (2021) provide a principled approach for estimating Shapley values under feature dependence, resulting in more stable and realistic attributions [33]. An even more scalable alternative is the Conditional Expectation Network proposed by Richman (2023), which uses a dedicated neural network to approximate the conditional expectation of the model output given any subset of input features. This approach avoids expensive sampling and is particularly well-suited for high-dimensional, structured input spaces like global weather fields [34].

Integrating such methods into GraphCast attribution could significantly improve the accuracy and physical plausibility of SHAP values, especially when investigating fine-scale interactions in multivariate fields.

7.3.5. Structure-Aware Interpretability

The current methods focus on variable-wise and spatial attribution, but deeper insight into model internals may be possible through graph-based or attention-based interpretability techniques. Methods such as Grad-CAM for GNNs, Edge Gate Values, GraphMask, or AttentionFlow could help identify critical paths of information propagation within GraphCast’s graph architecture. This could be especially relevant in revealing how localized meteorological signals are integrated or amplified through the model’s message-passing steps.

7.3.6. Integrating Domain Expertise

Finally, a deeper integration of meteorological expertise could strengthen both the selection of attribution targets and the interpretation of results. Collaborations with meteorologists and forecasters may therefore be a critical component of future work.

8. Conclusion

This study investigated the physical interpretability of the AI-based weather forecasting model GraphCast during the extreme wind event of Storm Zeynep on 18 February 2022. By combining standard forecast verification with a multi-level explainability framework, including model-agnostic feature importance, region-based occlusion, and gradient-based attribution, it was possible to assess whether the model’s predictions are both meteorologically plausible and interpretable.

The findings show that GraphCast, even in its reduced-resolution configuration, reproduces the spatial and temporal structure of the 10 m wind field with reasonable accuracy. Attribution analyses consistently identified wind speed and geopotential height as the primary predictors, particularly in dynamically active regions such as the Western Europe Jet Streak and the Storm Core. Vertical and spatial attribution patterns closely aligned with meteorological expectations, supporting the view that GraphCast implicitly encodes known physical relationships such as pressure gradients, jet-induced forcing, and low-level jet contributions.

Importantly, the results from region-based occlusion and integrated gradients were highly consistent, with both methods independently highlighting the Western Europe Jet Streak and the Storm Core as key drivers of surface wind speed. This agreement between model-agnostic perturbation methods and gradient-based attribution not only strengthens confidence in the physical plausibility of the findings but also demonstrates that the predefined meteorological regions align well with the data-driven structures revealed by integrated gradients.

Beyond confirming the model’s internal consistency for this specific event, this work contributes a generalizable interpretability framework for deep learning-based weather models under practical constraints. The proposed strategy, combining global, regional, and spatial attribution, balances methodological limitations with physical insight. Each method provides a different level of abstraction: global explainers highlight dominant drivers, occlusion isolates vertically and spatially relevant structures, and integrated gradients reveal coherent dynamical patterns. Together, they offer a more robust and complementary understanding than any single technique could provide, especially when internal model access is restricted.

Interpreting GraphCast also presents unique challenges. Unlike many GNNs, it operates as a large, multi-stage model with limited public access to internal components such as edge weights, node embeddings, or attention mechanisms. This restricts the use of many white-box interpretability tools commonly applied in graph-based models. Additionally, the model processes a vast number of features across space, pressure levels, and variables, complicating the use of local explainers like SHAP or LIME without careful feature aggregation. The framework developed in this thesis addresses these challenges by relying on model-agnostic and gradient-based methods, and by grouping features into meteorologically meaningful regions, making interpretation feasible despite limited access.

More broadly, this thesis highlights the growing importance of explainability in AI-based forecasting systems. As such models become increasingly central to operational meteorology, particularly under conditions of extreme weather, it becomes crucial to understand not just what the model predicts, but why. Explainable AI offers a pathway to bridge this gap, enabling forecasters and researchers to evaluate model trustworthiness, diagnose potential biases, and better communicate uncertainty in high-impact scenarios.

However, this study also highlights the limitations of current XAI methods in meteorology. Many techniques struggle with high-dimensional, correlated inputs and may introduce artifacts that complicate interpretation. Attribution results are often sensitive to design choices such as baseline selection or region definitions. While XAI provides useful insights, its outputs require careful interpretation and should be complemented by domain expertise. Future research should refine attribution tools and explore methods that reveal GraphCast’s internal structure, to ensure AI-based weather models remain both interpretable and operationally reliable.

References

- [1] K. Bi, L. Xie, H. Zhang, X. Chen, X. Gu, and Q. Tian, “Accurate medium-range global weather forecasting with 3D neural networks,” *Nature*, vol. 619, no. 7970, pp. 533–538, July 2023, doi: 10.1038/s41586-023-06185-3.
- [2] R. Lam *et al.*, “Learning skillful medium-range global weather forecasting,” *Science*, vol. 382, no. 6677, pp. 1416–1421, Dec. 2023, doi: 10.1126/science.adi2336.
- [3] “Wetter und Klima - Deutscher Wetterdienst - Thema des Tages - Orkan ZEYNEP - Ein regional historisches Naturereignis.” Accessed: May 13, 2025. [Online]. Available: https://www.dwd.de/DE/wetter/thema_des_tages/2022/2/19.html
- [4] *google-deepmind/graphcast*. (May 12, 2025). Python. Google DeepMind. Accessed: May 13, 2025. [Online]. Available: <https://github.com/google-deepmind/graphcast>
- [5] S. SADEGHI TABAS *et al.*, “GFS-Powered Machine Learning Weather Prediction: A Comparative Study on Training GraphCast with NOAA’s GDAS Data for Global Weather Forecasts,” 2025, doi: 10.25923/XD3Y-WY31.
- [6] Allen, Myles R., J. A. Kettleborough, and D. A. Stainforth, “Model error in weather and climate forecasting,” *ECMWF Predict. Weather Clim. Semin. Eur. Cent. Medium Range Weather Forecasts Read. UK*, pp. 279–304, 2002.
- [7] T. N. Palmer, G. J. Shutts, R. Hagedorn, F. J. Doblas-Reyes, T. Jung, and M. Leutbecher, “REPRESENTING MODEL UNCERTAINTY IN WEATHER AND CLIMATE PREDICTION,” *Annu. Rev. Earth Planet. Sci.*, vol. 33, no. 1, pp. 163–193, May 2005, doi: 10.1146/annurev.earth.33.092203.122552.
- [8] J. Pathak *et al.*, “FourCastNet: A Global Data-driven High-resolution Weather Model using Adaptive Fourier Neural Operators,” 2022, *arXiv*. doi: 10.48550/ARXIV.2202.11214.
- [9] I. Price *et al.*, “Probabilistic weather forecasting with machine learning,” *Nature*, vol. 637, no. 8044, pp. 84–90, Jan. 2025, doi: 10.1038/s41586-024-08252-9.
- [10] T. N. Kipf and M. Welling, “Semi-Supervised Classification with Graph Convolutional Networks,” Feb. 22, 2017, *arXiv*: arXiv:1609.02907. doi: 10.48550/arXiv.1609.02907.
- [11] S. Brody, U. Alon, and E. Yahav, “How Attentive are Graph Attention Networks?,” Jan. 31, 2022, *arXiv*: arXiv:2105.14491. doi: 10.48550/arXiv.2105.14491.

- [12] P. Veličković, G. Cucurull, A. Casanova, A. Romero, P. Liò, and Y. Bengio, “Graph Attention Networks,” Feb. 04, 2018, *arXiv*: arXiv:1710.10903. doi: 10.48550/arXiv.1710.10903.
- [13] J. Gilmer, S. S. Schoenholz, P. F. Riley, O. Vinyals, and G. E. Dahl, “Neural Message Passing for Quantum Chemistry,” 2017, *arXiv*. doi: 10.48550/ARXIV.1704.01212.
- [14] P. W. Battaglia *et al.*, “Relational inductive biases, deep learning, and graph networks,” Oct. 17, 2018, *arXiv*: arXiv:1806.01261. doi: 10.48550/arXiv.1806.01261.
- [15] A. B. Arrieta *et al.*, “Explainable Artificial Intelligence (XAI): Concepts, taxonomies, opportunities and challenges toward responsible AI,” *Inf. Fusion*, vol. 58, pp. 82–115, 2020.
- [16] R. Guidotti, A. Monreale, S. Ruggieri, F. Turini, D. Pedreschi, and F. Giannotti, “A Survey Of Methods For Explaining Black Box Models,” June 21, 2018, *arXiv*: arXiv:1802.01933. doi: 10.48550/arXiv.1802.01933.
- [17] M. D. Zeiler and R. Fergus, “Visualizing and Understanding Convolutional Networks,” in *Computer Vision – ECCV 2014*, vol. 8689, D. Fleet, T. Pajdla, B. Schiele, and T. Tuytelaars, Eds., in Lecture Notes in Computer Science, vol. 8689. , Cham: Springer International Publishing, 2014, pp. 818–833. doi: 10.1007/978-3-319-10590-1_53.
- [18] A. Fisher, C. Rudin, and F. Dominici, “All Models are Wrong, but Many are Useful: Learning a Variable’s Importance by Studying an Entire Class of Prediction Models Simultaneously,” *J. Mach. Learn. Res. JMLR*, vol. 20, p. 177, 2019.
- [19] C. Molnar, “Interpretable Machine Learning,” in *A Guide for Making Black Box Models Explainable*, 3rd ed., 2025. Accessed: May 20, 2025. [Online]. Available: <https://christophm.github.io/interpretable-ml-book>
- [20] Lloyd S Shapley, “A value for n-person games.,” *Contrib. Theory Games*, no. 2.28, pp. 307–317, 1953.
- [21] E. Mosca, F. Szigeti, S. Tragianni, D. Gallagher, and G. Groh, “SHAP-Based Explanation Methods: A Review for NLP Interpretability”.
- [22] S. M. Lundberg and S.-I. Lee, “A Unified Approach to Interpreting Model Predictions,” in *Advances in Neural Information Processing Systems*, I. Guyon, U. V. Luxburg, S. Bengio, H. Wallach, R. Fergus, S. Vishwanathan, and R. Garnett, Eds., Curran Associates, Inc., 2017. [Online]. Available:

https://proceedings.neurips.cc/paper_files/paper/2017/file/8a20a8621978632d76c43dfd28b67767-Paper.pdf

- [23] G. Montavon, W. Samek, and K.-R. Müller, “Methods for interpreting and understanding deep neural networks,” *Digit. Signal Process.*, vol. 73, pp. 1–15, Feb. 2018, doi: 10.1016/j.dsp.2017.10.011.
- [24] M. Ancona, E. Ceolini, C. Öztireli, and M. Gross, “Towards better understanding of gradient-based attribution methods for Deep Neural Networks,” Mar. 07, 2018, *arXiv*: arXiv:1711.06104. doi: 10.48550/arXiv.1711.06104.
- [25] K. Simonyan, A. Vedaldi, and A. Zisserman, “Deep Inside Convolutional Networks: Visualising Image Classification Models and Saliency Maps,” Apr. 19, 2014, *arXiv*: arXiv:1312.6034. doi: 10.48550/arXiv.1312.6034.
- [26] A. Shrikumar, P. Greenside, and A. Kundaje, “Learning Important Features Through Propagating Activation Differences,” Oct. 12, 2019, *arXiv*: arXiv:1704.02685. doi: 10.48550/arXiv.1704.02685.
- [27] M. Sundararajan, A. Taly, and Q. Yan, “Axiomatic Attribution for Deep Networks,” *Int. Conf. Mach. Learn. PMLR*, pp. 3319–3328, 2017.
- [28] “ERA5 hourly data on single levels from 1940 to present.” Accessed: May 14, 2025. [Online]. Available: <https://cds.climate.copernicus.eu/datasets/reanalysis-era5-single-levels?tab=overview>
- [29] B. Mühr, L. Eisenstein, J. G. Pinto, P. Knippertz, S. Mohr, and M. Kunz, “CEDIM Forensic Disaster Analysis Group (FDA): Winter storm series: Ylenia, Zeynep, Antonia (int: Dudley, Eunice, Franklin)-February 2022 (NW & Central Europe),” 2022, Accessed: May 26, 2025. [Online]. Available: <https://publikationen.bibliothek.kit.edu/1000143470>
- [30] H. F. Dacre and J. G. Pinto, “Serial clustering of extratropical cyclones: a review of where, when and why it occurs,” *Npj Clim. Atmospheric Sci.*, vol. 3, no. 1, p. 48, Nov. 2020, doi: 10.1038/s41612-020-00152-9.
- [31] J. R. Holton and G. J. Hakim, *An introduction to dynamic meteorology*, Fifth edition. Amsterdam: Academic Press, 2013.
- [32] K. A. Emanuel, “An Air-Sea Interaction Theory for Tropical Cyclones. Part I: Steady-State Maintenance,” *J. Atmospheric Sci.*, vol. 43, no. 6, pp. 585–605, Mar. 1986, doi: 10.1175/1520-0469(1986)043<0585:AASITF>2.0.CO;2.

- [33] K. Aas, M. Jullum, and A. Løland, “Explaining individual predictions when features are dependent: More accurate approximations to Shapley values,” Feb. 06, 2020, *arXiv*: arXiv:1903.10464. doi: 10.48550/arXiv.1903.10464.
- [34] R. Richman and M. V. Wüthrich, “Conditional expectation network for SHAP,” July 20, 2023, *arXiv*: arXiv:2307.10654. doi: 10.48550/arXiv.2307.10654.
- [35] P.-J. Kindermans *et al.*, “The (Un)reliability of saliency methods,” 2017, *arXiv*. doi: 10.48550/ARXIV.1711.00867.
- [36] A. Ghorbani, A. Abid, and J. Zou, “Interpretation of Neural Networks Is Fragile,” *Proc. AAAI Conf. Artif. Intell.*, vol. 33, no. 01, pp. 3681–3688, July 2019, doi: 10.1609/aaai.v33i01.33013681.

Appendix

Table A1: Variables and level in GraphCast, boldfaced levels are included in GraphCast_small [2]

Category	Feature
Atmospheric variables	Temperature U component of wind V component of wind Geopotential Specific humidity Vertical wind speed
Surface variables	2-meter temperature 10-meter u wind component 10-meter v wind component Mean sea-level pressure Total precipitation
Pressure level (in hPa)	1, 2, 3, 5, 7, 10, 20, 30, 50 , 70, 100 , 125, 150 , 175, 200 , 225, 250 , 300 , 350, 400 , 450, 500 , 550, 600 , 650, 700 , 750, 775, 800, 825, 850 , 875, 900, 925 , 950, 975, 1000

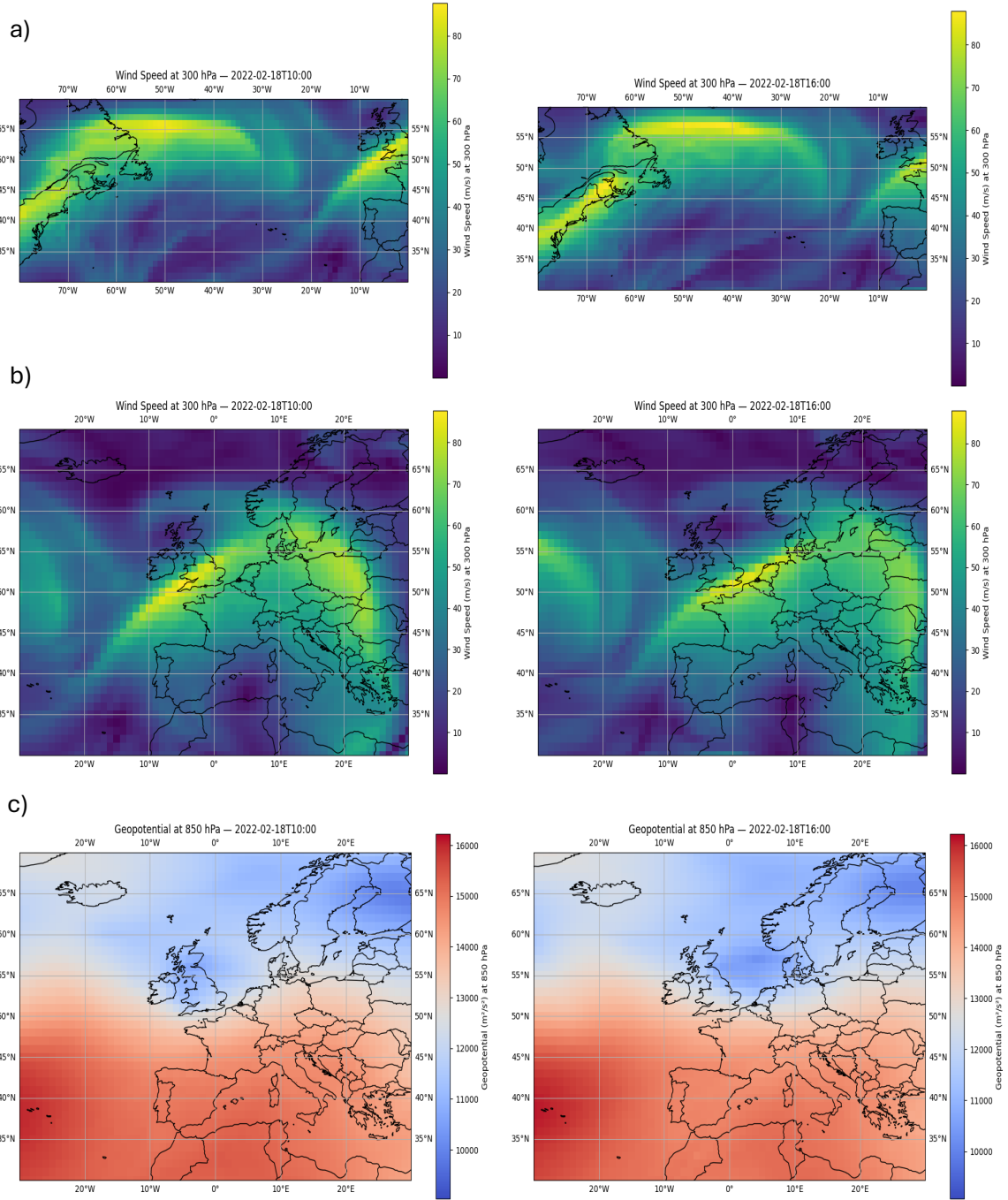


Figure A1: Visualization of meteorological drivers used for spatial feature selection

As input served the ERA5 reanalysis data on February 18th, 2022, at 10:00 and 16:00 UTC. (a) Wind speed at 300 hPa over the North, showing the Atlantic Jet Stream as a yellow band between 40°N and 60°N. (b) Wind speed at 300 hPa over Western Europe, highlighting the upper-level jet stream across the storm region. (c) Geopotential at 850 hPa, used to locate the Storm Core via the region of minimum geopotential and to illustrate the pressure gradient between northern and southern Europe.

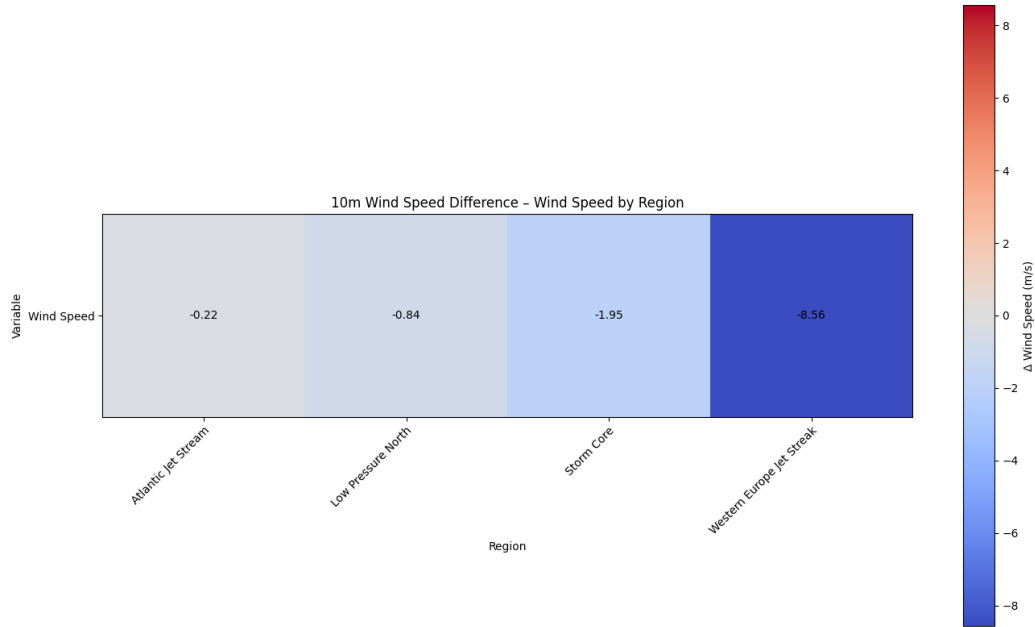


Figure A2: Occlusion of different regions for the atmospheric variable wind speed

Mean absolute change in predicted 10 m wind speed over northern Germany (53°N–55°N, 8°E–10°E) due to occlusion of wind speed in different meteorological regions. Regions were occluded using a climatological baseline.

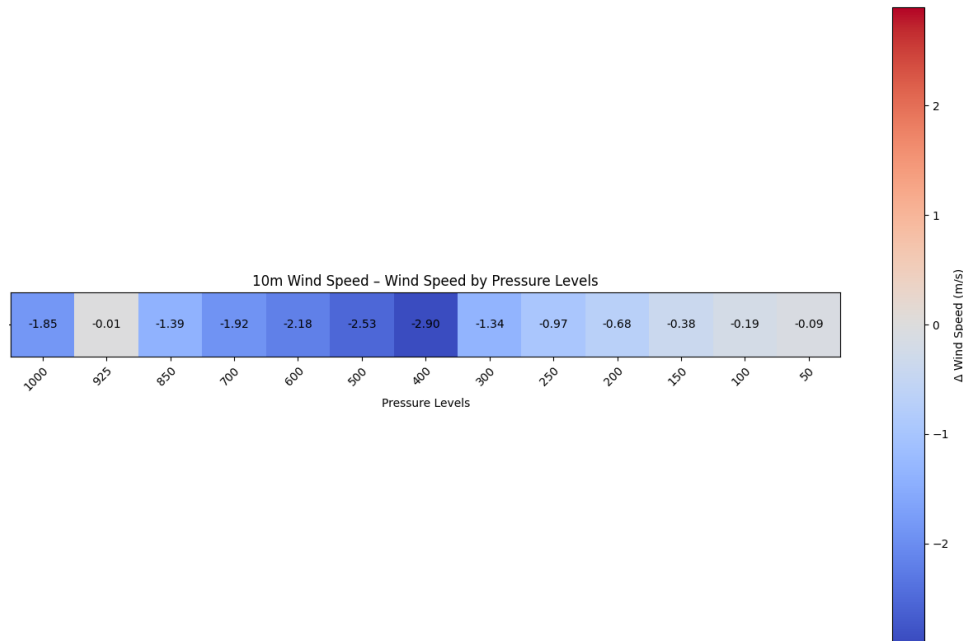


Figure A3: Occlusion of different pressure levels for the atmospheric variable wind speed

Mean absolute change in predicted 10 m wind speed over northern Germany resulting from occlusion of wind speed at different pressure levels. Pressure levels were occluded using a climatological baseline.

Code Repository

The full source code developed for this thesis, including data handling, forecast generation, and explainability analysis, is publicly available at:

<https://github.com/EStaELisa/graphcast-xai>

This repository contains all scripts, configuration files, and Jupyter notebooks required to reproduce the results presented in this work.